

→ Operational Count

$$T(n) = C_{op} C(n)$$

→ Polynomial Evaluation

$$\Rightarrow \text{no. of additions} = n$$

$$\Rightarrow \text{no. of multiplication} = 2n$$

$$\Rightarrow \text{Hence, time complexity: } O(n)$$

→ Matrix Multiplication

$$\Rightarrow \text{Key Op: Multiplication}$$

$$\Rightarrow \text{time complexity:}$$

$$O(mnp)$$

→ Rank of an element in a sequence

$$\Rightarrow \text{Key Operation} = \text{Comparison}$$

$$\Rightarrow i = n-1,$$

$$1+2+3+\dots+n(n-1) \left[\text{Series} \propto \frac{n(n-1)}{2} \right]$$

$$\text{total no. of comparisons} = \frac{(n-1)n}{2}$$

$$\Rightarrow \text{Time complexity: } O(n^2)$$

→ Rank Sort

$$\Rightarrow \text{time complexity: } O(n^2)$$

$$\Rightarrow \text{Rearrangement time: } O(n) \text{ times}$$

→ Insertion Sort

$$\Rightarrow \text{Since } j = i-1 \Rightarrow 1+2+3+\dots+n(n-1)$$

$$\Rightarrow \frac{n(n-1)}{2} \quad (\approx \text{by series})$$

$$\Rightarrow \text{Time complexity: } \frac{n(n-1)}{2} \approx n^2 = O(n^2)$$

→ Step Count Method

$$\text{Note: } \log(n) = 2^k$$

Ultimate

→ Recursive Algorithm

$$\Rightarrow \text{time complexity: } O(n)$$

$$\Rightarrow \text{Base case: } T_{sum}(0) = 2$$

$$\hookrightarrow T_{sum}(n) = T_{sum}(n-1) + 2$$

$$\Rightarrow \text{Calculating Complexity, } T_{sum}(n) = 2 + T_{sum}(n-1)$$

$$= 2 + 2 + T_{sum}(n-2)$$

$$= 4 + T_{sum}(n-2)$$

$$\vdots$$

$$\begin{aligned} & \vdots \\ & = 2n + T_{sum}(0) \\ & \text{ } T_{sum}(0) : \text{Base case} \\ & = 2n + 2, n \geq 0 \end{aligned}$$

Ex: $T(n) = T(n-1) + cn$, where base case $T(0) = 1$ [Here Test case goes upto $n-1$]

Now Simplifying,

$$\begin{aligned}
 T(n) &= T(n-1) + n \\
 &= T(n-2) + 2n \quad \leftarrow n+n \\
 &= T(n-3) + 3n \quad \leftarrow n+2n \\
 &= \vdots \quad \leftarrow 1+2+3+\dots+\frac{n(n-1)}{2} \\
 &= T(n-n) + n^2 \quad \Rightarrow n^2 \\
 &= T(0) + n^2 \\
 &= 1 + n^2 \\
 &\approx \underline{O(n^2)}
 \end{aligned}$$

→ Recursive Algorithm (contin...)

⇒ Base Case: $T(0) = 2$

$$\begin{aligned}
 \Rightarrow T(n) &= T(n-1) + 3 \quad (3 \times 1) \\
 &= T(n-2) + 6 \quad (1 \times 2) \quad (3 \times 2) \\
 &= T(n-3) + 9 \quad (1 \times 3) \quad (3 \times 3) \\
 &= T(n-n) + 3n \\
 &= 2 + 3n \\
 &\approx \underline{O(n)}
 \end{aligned}$$

} RSeq. Search()

Q2/1/26

Ex: $C(n) = \frac{1}{2} n(n-1) = \frac{1}{2} n^2 - \frac{1}{2} n \approx \frac{1}{2} n^2$

→ Recursive Algorithm (cont...) where for loop $i++$ $i \leq 2$

⇒ Base Case: $T(0) = 2$

⇒ $T(n) = T(n-1) + \log n$

$= T(n-2) + \log n + \log(n-1)$

$= T(n-3) + \log n + \log(n-1) + \log(n-2)$

$\vdots$

$= T(n-n) + \log n + \log(n-1) + \dots + \log(1)$

$= T(0) + \log n^2$

$= 1 + \log n^2$

$\approx \underline{O(n \log n)}$

$$\begin{aligned}
 \text{Ex: } T(n) &= T(n-1) + n^2, \quad T(0) = 1 \\
 &= T(n-1) + 2n^2 \\
 &= T(n-1) + 3n^2 \\
 &\vdots \\
 &= T(n-n) + n \times n^2 \\
 &= T(0) + n^3 \\
 &= 1 + n^3 \\
 &\approx \underline{\underline{O(n^3)}}
 \end{aligned}$$

→ Best, Worst & Average Case time complexity

→ SeqSearch()

$$\begin{aligned}
 \Rightarrow \text{no. of steps: } 4 & \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{Best case} \\
 &= \underline{\underline{O(1)}}
 \end{aligned}$$

$$\begin{aligned}
 \Rightarrow \text{no. of steps: } 1, n+2, n+3, \dots & \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{Worst case} \\
 &= \underline{\underline{O(n)}}
 \end{aligned}$$

$$\Rightarrow \text{no. of steps: } 4$$

$$= \frac{1+2+3+\dots+n}{n}$$

$$= \frac{n(n+1)/2}{n} = \frac{n+1}{2}$$

$$\approx \frac{n}{2}$$

→ Avg. calculation to find mean

$$\frac{n}{2} + \frac{n}{2} + 1 + \frac{n}{2}$$

$$+1$$

$$+1$$

$$\frac{3n+4}{2}$$

$$\text{Now, } a = 3/2 \text{ \& } b = 4$$

$$T(n) = an + b \Rightarrow \underline{\underline{O(n)}}$$

Avg. case

→ Selection Sort

$$\Rightarrow (n-1)(n-2) + \dots + (n-n)$$

$$\Rightarrow n(n-1) + \dots + 0$$

$$\Rightarrow n(n-1)$$

$$\Rightarrow n^2 - n \approx \underline{\underline{O(n^2)}}$$

Valid for Best, Avg. & Worst Case
[sorted/unsorted]

Ex: best case :
 worst case :
 Avg. case :

Problem 1:

```
int i, j, k = 0;
for (i = n/2; i <= n; i++) {
  for (j = 2; j <= n; j = j * 2) {
    k = k + n/2;
  }
}
```

1 — $i = \frac{n}{2} \text{ to } n = n - \frac{n}{2} + 1 = \frac{n}{2} + 1 \approx \frac{n}{2}$

1 — $j = 2; j \leq n; \text{ no. of iterations} = \log n$

$\therefore \text{no. of iterations} = \frac{n}{2} \log n$

steps = 2

best case :
 worst case :
 Avg. case : $\left\{ \frac{n}{2} \log n \right\}$

Complexity : $O(n \log n)$

Problem 2:

```
int a = 0; i = N;
while (i > 0) {
  a += i;
  i /= 2;
}
```

steps = 2

1 — $\log n$
 1 — 2

$\therefore T(n) = 2 \log n$

best case :
 worst case :
 Avg. case : $\left\{ 2 \log n \right\}$

Complexity : $O(\log n)$

ex 11/26
 →

Sequential Search

→ Best case : $O(1)$
 → Worst case : $O(n)$
 → Avg case : $O(n)$

→ Binary Search

→ Best case : $O(1)$
 → Worst case : $O(\log n)$
 → Avg case : $O(\log n)$

for balanced tree
 [or $O(h)$: height of the tree]
 → $O(n)$: skewed trees only

→ Brute Force

→ selection sort (sorted w. the smallest no.)

key op. : comparison & swap

$$\text{comparison: } (n-1)(n-2) \dots (n-n) \\ \Rightarrow \frac{n(n+1)}{2}$$

$$\text{swaps: } \underline{\underline{(n-1)}}$$

time complexity : $O(n^2)$

space complexity : $O(1)$

stability : No (Not Adaptive)

why not stable?



→ bubble sort (sorted comparing w. the largest no.)

→ no. of passes : $(n-1)$

$$\text{comparison: } (n-1)(n-2) \dots (n-n) \\ \Rightarrow \frac{n(n+1)}{2}$$

$$\text{swaps: } \frac{n(n+1)}{2}$$

time complexity : $O(n^2)$

space complexity :

stability : Yes

27/1/26

→ String Matching

n → string length

m → substring / pattern length

$P[j] = T[i+j]$ → comparison b/w string & substring

$j = m$ → string is matched

Time Complexities :- Best Case :

worst case : $O(n \cdot m)$

Avg. case : $O(n+m)$

→ Exhaustive search

→ Problems:-

- ① Travelling Salesman Problem
- ② Knapsack Problem
- ③ Assignment Problem

① Travelling salesman Problem (minimization problem: optimal sol.)

2 8 1 7 — 18

2 3 1 5 — 11

2 5 8 3 — 18

2 7 1 5 — 15

2 8 1 3 — 14

5 8 3 7 — 23

Time Complexity:-

$$O(n-1)!$$

② Knapsack Problem (maximization problem: search profit)

Q $W = 8$

	Item(i)	Weight(w_i)	Value(v_i)
	1	2	\$1
	2	3	\$2
	3	4	\$8
	4	5	\$6
{1}	2	\$1	
{2}	3	\$2	
{3}	4	\$8	
{4}	5	\$6	

{1, 2}	5	\$3	W = 8
{1, 3}	6	\$9	
{1, 4}	7	\$7	
{2, 3}	7	\$10	Optimal solution
{2, 4}	<u>8</u>	<u>\$8</u>	(feasible, but less profit)
{3, 4}	9 x	<u>\$14</u>	} not feasible
{1, 2, 3}	9 x	<u>\$11</u>	
{1, 2, 4}	10	\$9	
{2, 3, 4}	12 x	<u>\$16</u>	} not feasible
{1, 2, 3, 4}	14 x	<u>\$17</u>	

Time Complexity: 2^n

Ex:	W = 10	Item (i)	Weight (w _i)	Value (v _i)
		1	3	\$50
	$2^4 = 16$	2	4	\$30
		3	2	\$60
		4	5	\$40

{1}	3	\$50
{2}	4	\$30
{3}	2	\$60
{4}	5	\$40
{1, 2}	7	\$80
{1, 3}	5	\$110
{1, 4}	8	\$90
{2, 3}	6	\$90
{2, 4}	9	\$70
{3, 4}	7	\$100
{1, 2, 3}	9	\$140
{1, 2, 4}	12 x	\$120
{2, 3, 4}	11 x	\$130
{1, 2, 3, 4}	14 x	\$180

③ Assignment Problem

Q)

	T_1	T_2	T_3
P_1	4	7	2
P_2	3	5	8
P_3	9	1	6

$$n! = 3! = \underline{\underline{6}}$$

Time complexity = $O(n!)$

6 subsets

$$4 + 5 + 6 = 15$$

$$4 + 8 + 1 = 13$$

$$3 + 7 + 6 = 16$$

$$\boxed{3 + 2 + 1 = 6}$$

$$9 + 2 + 5 = 16$$

$$9 + 7 + 8 = 24$$

Optimal solution

21/1/26

→ Dfs & Bfs

→ Dfs [preorder tree traversal, Depth first search]

→ Bfs [level order tree traversal, Breadth first search]

$$O(n + e)$$

where $n = |V|$ & $e = |E|$

→ Decrease & Conquer

① Decrease by a constant

② Decrease by a constant factor

③ variable size decrease

3 Topics :- ① Insertion sort [Decrease by constant]

Ex: Array, $A = 5 \ 7 \ 11 \ 13 \ 20 \ 22$

Insert $x = 15$

→ $5 \ 7 \ 11 \ 13 \ 20 \ 15 \ 22$

→ $5 \ 7 \ 11 \ 13 \ 15 \ 20 \ 22$

→ $5 \ 7 \ 11 \ 13 \ 15 \ 20 \ 22$ [sorted!]

Worst case: $O(n^2)$

Avg case: $O(n^2)$

Best case: $O(n)$ [$\Rightarrow n-1 \Rightarrow n$]

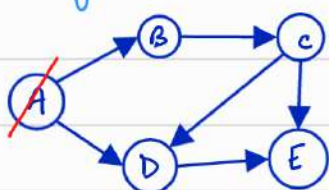
3/2/26

② Binary search [Decrease by Constant factor]

Complexity: $O(\log n)$

③ Topological Sorting [Variable size decrease]

Method 1: sorting by cancelling elements one by one by its indegree = 0.



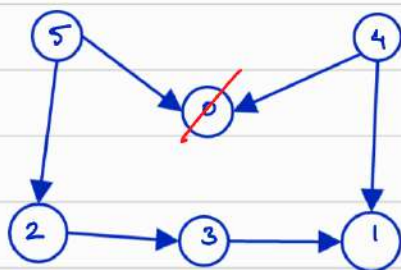
$\Rightarrow$ T-sort = A F B C D E

follows $\Rightarrow$ Time Complexity: $O(|V| + |E|)$

Adj. Matrix: $O(n^2)$

Adj. list: $O(n^2 + 1)$

Method 2: Topological sort by DFS



$i=0$,

T	F	F	F	F	F
0	1	2	3	4	5

 no out degree, hence true

Therefore,

		5
		4
	$\Rightarrow$	2
	continue till $i=5$	3
		1
0		0

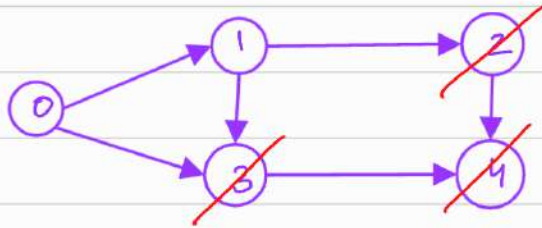
final output:- popping from top to down we get,

5 4 2 3 1 0

Time complexity: $O(|V| + |E|)$

Space complexity: $O(|V|)$

Ex:

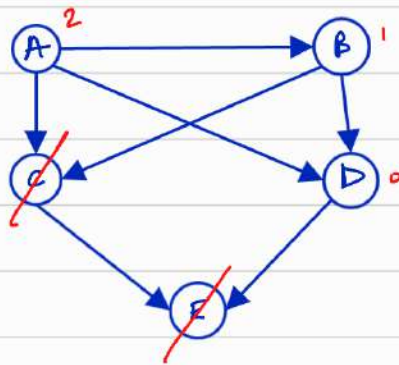


A:

pushing to stack $\Rightarrow$ 0
1
2
3
4

; popping to gain output $\Rightarrow$ 0 1 2 3 4

Q:



pushed stack

ECDBA } 2 diff. o/p
EDCBA }

$\therefore$ output :- A B D C E

A B C D E

Q:

Prereq. for CS 143 $\rightarrow$ MATH 126 & CS 142

CS 373 $\rightarrow$ CS 874 is CS 143

CS 417 $\rightarrow$ CS 373

Mostly not Asked

Method 3 : Critical Path : Calculating duration

E.S $\rightarrow$ earliest start (same for L.S) TF $\rightarrow$ Total float

E.F $\rightarrow$ earliest finish (same for L.F)

D $\rightarrow$ Duration

ES	D	LS
EF	TF	LF

* E.F for one node is E.S for the next node.

* $E.F = E.S + D$

* $L.S = L.F - D$

* $T.F = L.F - E.F$

4/2/26



Divide and conquer

$\rightarrow$ Problems :-

① Merge sort

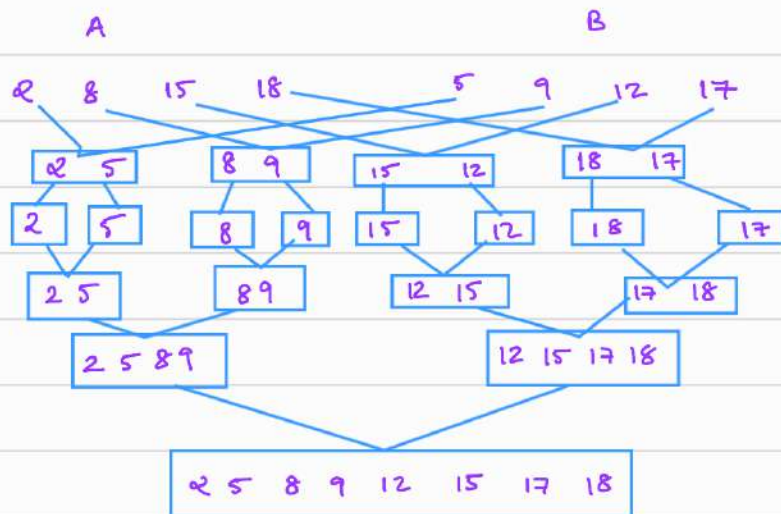
② Quick sort

③ Matrix Multiplication (Strassen's Algo.)

④ Multiplication of large no's.

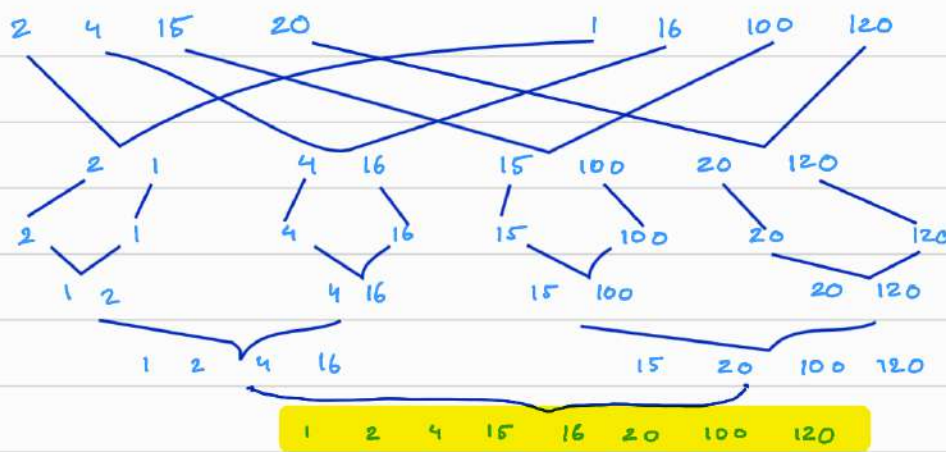
① Merge sort

Ex :



Time complexity: $O(p+q) = O(n)$ comparisons [for merging]

$O(n \log n) \rightarrow$ best case



$$T(n) = 2T(n/2) + cn ; T(1) = 0 \text{ for constant } c$$

$$= 2T(n/4) + cn$$

$$= 2T(n/8) + 2cn$$

$$= 2T(n/16) + 3cn$$

⋮

$$= 2T(n/2^k) + kcn$$

$$= 2T(n/2^k) + c(\log n)(n)$$

$$= 2T(n/n) + c(\log n)(n)$$

$$= 2T(1) + c(\log n)(n)$$

$$= 2(0) +$$

$$= 0 + c(\log n)(n)$$

$$= \underline{\underline{O(n \log n)}}$$

$$\therefore 2^k = n ; k = \log n$$

$$10 \leq \left(\frac{n}{3}\right)$$

$$a = 10$$

$$b = 3$$

$$n^{\log_3 10}$$

$$n^{2.10}$$

$$a = 128$$

$$n^7$$

→ Master's Theorem

$$\text{If } a < b^d \Rightarrow T(n) \in O(n^2)$$

$$\text{If } a = b^d \Rightarrow T(n) \in O(n^d \log n)$$

$$\text{If } a > b^d \Rightarrow T(n) \in O(n^{\log_b a})$$

5/2/26

② Quick Sort

→ Partition of n elements into 3 parts :- left, middle & right

→ All elements are $\leq$ pivot in the left

→ All elements are $\geq$ pivot in the right

a)

6	2	8	5	11	10	4	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

 [partition method]

A:

pivot = 6 $\Rightarrow$

2	5	4	1	3		6		7	9	10	11	8
---	---	---	---	---	--	---	--	---	---	----	----	---



pivot = 2 $\Rightarrow$

1					2		3	4	5
---	--	--	--	--	---	--	---	---	---

pivot = 8 $\Rightarrow$

7		8		11	10	9
---	--	---	--	----	----	---

pivot = 5 $\Rightarrow$

1	2	3	4	5
---	---	---	---	---

pivot = 11 $\Rightarrow$

7	8	9	10	11
---	---	---	----	----

pivot = 10 $\Rightarrow$

7	8	9	10	11
---	---	---	----	----

pivot = 4 $\Rightarrow$

1	2	3	4	5
---	---	---	---	---

pivot = 1 $\Rightarrow$

1	2	3	4	5
---	---	---	---	---

pivot = 9 $\Rightarrow$

7	8	9	10	11
---	---	---	----	----

pivot = 7 $\Rightarrow$

7	8	9	10	11
---	---	---	----	----

pivot = 3 $\Rightarrow$

1	2	3	4	5
---	---	---	---	---

Ex:

same as ↑ But using Swapping method.

6	2	8	5	11	10	4	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

Step:

6	2	8	5	11	10	4	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p > i$ & $p > j$

6	2	8	5	11	10	4	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p < i$ & $p < j$

6	2	8	5	11	10	4	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p > i$ & $p < j$

6	2	8	5	11	10	4	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p < i$ & $p > j$ (SWAP)

6	2	8	5	1	10	4	11	9	7	3
---	---	---	---	---	----	---	----	---	---	---

p i j

$p < i$ & $p > j$ (SWAP)

6	2	8	5	11	4	10	1	9	7	3
---	---	---	---	----	---	----	---	---	---	---

p j i

$p < i$ & $p > j$ (SWAP w. p)

4	2	8	5	11	10	6	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p > i$ & $p > j$

4	2	8	5	11	10	6	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p < i$ & $p < j$

4	2	8	5	11	10	6	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p < i$ & $p > j$ (SWAP)

4	2	8	9	11	10	6	1	5	7	3
---	---	---	---	----	----	---	---	---	---	---

p i j

$p < i$ & $p > j$ (SWAP)

4	2	8	5	1	10	6	11	9	7	3
---	---	---	---	---	----	---	----	---	---	---

p i j

$p < i$ & $p < j$

4	2	8	5	1	10	6	11	9	7	3
---	---	---	---	---	----	---	----	---	---	---

p j i

$p < i$ & $p < j$

4	2	8	5	1	10	6	11	9	7	3
---	---	---	---	---	----	---	----	---	---	---

p j i

$p < i$ & $p > j$ (SWAP w. p)

1	2	8	5	4	10	6	11	9	7	3
---	---	---	---	---	----	---	----	---	---	---

p i j

7/2/20

Average Case: Quick Sort

$$T(n) = \frac{(n+1) (T(L) + T(R))}{n}$$

$$T(n) = \frac{(n+1) + 2 [T(0) + T(1) + \dots + T(n-1)]}{n}$$

$$nT(n) = n(n+1) + 2 [T(0) + T(1) + \dots + T(n-1)] \quad \text{--- ①}$$

$$(n-1)T(n-1) = n(n-1) + 2 [T(0) + T(1) + \dots + T(n-2)] \quad \text{--- ②}$$

$$nT(n) - (n-1)T(n-1) = n(n+1) - n(n-1) + 2T(n-1)$$

$$nT(n) = \cancel{n^2} + n - \cancel{n^2} + n + 2T(n-1) + (n-1)T(n-1)$$

from
David Langner - DAA.pdf

③ Matrix Multiplication

Time Complexity: n^3

for 4×4 matrix: $- T\left(\frac{n}{2}\right) \times T\left(\frac{n}{2}\right)$

$$\text{We do, } T(n) = \overset{\text{no. of mul.}}{8} T\left(\frac{n}{2}\right) + \underset{\text{a x a}}{cn^2}$$

$\Rightarrow a=8; b=2$; therefore applying Master's theorem,

$$\Rightarrow a > b; O(n^{\log_b a})$$

$$\Rightarrow O(n^{\log_2 8}) \quad [\because \log_2 8 = 3]$$

$$\Rightarrow \underline{O(n^3)}$$

$\rightarrow$ Strassen Matrix

Time Complexity: $O(n^{2.81}) \rightarrow$ from $T(n) = 7T\left(\frac{n}{2}\right) + cn^2$

$$D = A_1(B_2 - B_4)$$

$$C_1 = E + I + J - G$$

$$E = A_1(B_3 - B_1)$$

$$C_2 = D + G$$

$$F = (A_3 + A_4)B_1$$

$$C_3 = E + F$$

$$G = (A_1 + A_2)B_4$$

$$C_4 = D + H + J - F$$

$$H = (A_3 - A_1)(B_1 + B_2)$$

$$I = (A_2 - A_4)(B_1 + B_4)$$

$$J = (A_1 + A_4)(B_1 + B_4)$$

10/2/26

Q

$$\begin{bmatrix} 1_{A_1} & 2_{A_2} \\ 3_{A_3} & 4_{A_4} \end{bmatrix} \begin{bmatrix} 5_{B_1} & 6_{B_2} \\ 7_{B_3} & 8_{B_4} \end{bmatrix} = ?$$

A:

$$D = A_1(B_2 - B_4) = 1(6 - 8) = -2$$

$$E = A_4(B_3 - B_1) = 4(7 - 5) = 8$$

$$F = (A_3 + A_4)B_1 = 5(3 + 4) = 35$$

$$G = (A_1 + A_2)B_4 = 8(1 + 2) = 24$$

$$H = (A_3 - A_1)(B_1 + B_2) = (3 - 1)(5 + 6) = 22$$

$$I = (A_2 - A_4)(B_1 + B_4) = (2 - 4)(7 + 8) = -30$$

$$J = (A_1 + A_4)(B_1 + B_4) = (1 + 4)(5 + 8) = 65$$

$$C_1 =$$

$$C_2 =$$

$$C_3 =$$

$$C_4 =$$

$$\therefore \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix} = \begin{bmatrix} & \\ & \end{bmatrix}$$

→ Master's Theorem

$$\text{If } a < b^d \Rightarrow T(n) \in O(n^2)$$

$$\text{If } a = b^d \Rightarrow T(n) \in O(n^d \log n)$$

$$\text{If } a > b^d \Rightarrow T(n) \in O(n^{\log_b a})$$

$$Q) a = 10, b = 3, d =$$

$$a > b \Rightarrow O(n^{\log_b a}) \Rightarrow O(n^{\log_3 10}) \Rightarrow O(n^{2.10})$$

$$Q) a = 128, b = 2, d =$$

$$a > b \Rightarrow O(n^{\log_b a}) \Rightarrow O(n^{\log_2 128}) \Rightarrow O(n^7)$$

④

Large Integer Multiplication

$$x = x_1 \cdot x^{n/2} + x_0$$

$$y = y_1 \cdot x^{n/2} + y_0$$

$x_1 \rightarrow$ high half ; $x_0 \rightarrow$ low half

$$z = x \cdot y = x_1 y_1 \cdot x^n + (x_0 y_1 + x_1 y_0) x^{n/2} + x_0 y_0$$

Time Complexity : $T(n) = 4T\left(\frac{n}{2}\right) + O(n)$

MT by substitution we get,

$$O(n^{\log_b a}) = O(n^{\log_2 4})$$

$$= \underline{O(n^2)}$$

$\rightarrow$ Karatsuba's Method

let,

$$z_0 = x_0 y_0$$

$$z_2 = x_1 y_1$$

$$z_1 = (x_0 + x_1) \cdot (y_0 + y_1) - z_0 - z_2$$

$$= x_0 y_1 + x_1 y_0$$

$$z = z_2 x^n + z_1 x^{n/2} + z_0$$

Q)

$$12 \mid 34 \times 43 \mid 21$$

$$z_0 = 34 \times 43 = \underline{714}$$

$$x_0 = 34 ; y_0 = 43$$

$$z_2 = 12 \times 21 = \underline{516}$$

$$x_1 = 12 ; y_1 = 21$$

$$z_1 = (12 + 34) \cdot (43 + 21) - 714 - 516$$

$$= 46 \cdot 64 - 714 - 516$$

$$= 2944 - 714 - 516$$

$$= \underline{1714}$$

$$z = 516 \times 10^4 + 1714 \times 10^2 + 714$$

$$= \underline{5682114}$$

Q) 2845×0678

A: $\therefore x_0 = 45; x_1 = 28; y_0 = 78; y_1 = 6$

$$z_0 = 45 \times 78 = \underline{\underline{3510}}$$

$$z_2 = 28 \times 6 = \underline{\underline{138}}$$

$$\begin{aligned} z_1 &= (28 + 45)(78 + 6) - z_0 - z_2 \\ &= 5712 - 3510 - 138 \\ &= \underline{\underline{2064}} \end{aligned}$$

$$\begin{aligned} z &= (138 \times 10^4) + (2064 \times 10^2) + 3510 \\ &= \underline{\underline{1589910}} \end{aligned}$$

$$\begin{aligned} &653214 \times \underline{\underline{32410}} \\ \Rightarrow &\underline{\underline{653}} \underline{\underline{214}} \times \underline{\underline{032}} \underline{\underline{410}} \end{aligned}$$

Q) 653214×32410

A: $m=3$ [Dividing by 3 digits]
 $653 \mid 214 \quad \underline{\underline{032}} \mid 410$

$$z_0 = 653 \times 32 = \underline{\underline{20896}}$$

$$z_2 = 214 \times 410 = \underline{\underline{87740}}$$

$$\begin{aligned} z_1 &= (653 + 214)(32 + 410) - z_0 - z_2 \\ &= (867)(442) - z_0 - z_2 \\ &= 383214 - 20896 - 87740 \\ &= \underline{\underline{274578}} \end{aligned}$$

$$\begin{aligned} z &= z_0 \times 10^{\overset{(2m)}{6}} + z_1 \times 10^{\overset{(m)}{3}} + z_2 \\ &= 20896 \times 10^6 + 274578 \times 10^3 + 87740 \\ &= \underline{\underline{2.117076574 \times 10^{11}}} \end{aligned}$$

11/2/26
→

Transform & Conquer

① Presorting

Worst case efficiency: $O(n^2)$

If quadratic sorting algo's. then,
sorting time: $O(n^2)$

Overall algo.: $O(n^2)$

If efficient sorting algo's. (like Merge/Heap Sort) then,

sorting time: $O(n \log n)$

checking steps: $O(n)$

Overall complexity: $O(n \log n)$

Ex: 5, 1, 5, 7, 6, 5, 7 and mode = 5

Input: array A of n integers

Output: mode of array

```
for (i=0; i<n; i++) {  
    for (j=i+1; j<n; j++) {  
        if (A[i] > A[j+1]) {  
            temp = A[i];  
            A[j] = A[j+1];  
            A[j+1] = temp;  
        }  
    }  
}
```

} using
Bubble
Sort

mode = A[0];

```
for (i=1; i<n; i++) {  
    if (A[i] == A[i-1]) {
```

Ex: Searching problem

→ n comparisons in worst case then,
we apply binary search which req. only
 $\log(n+1)$: comparisons

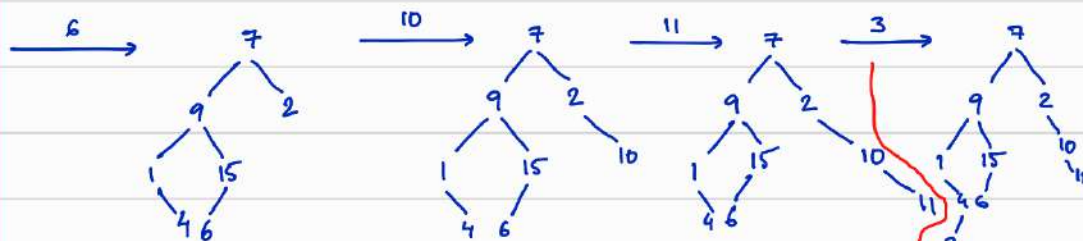
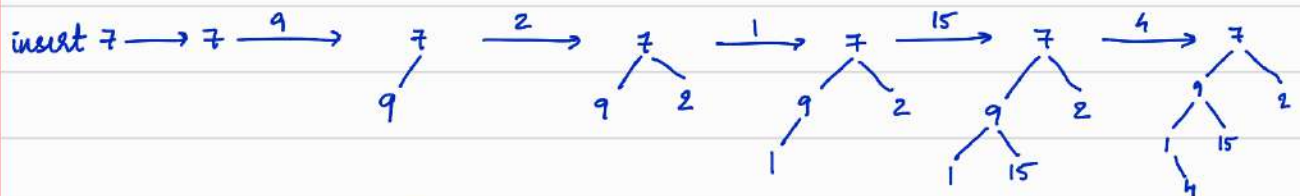
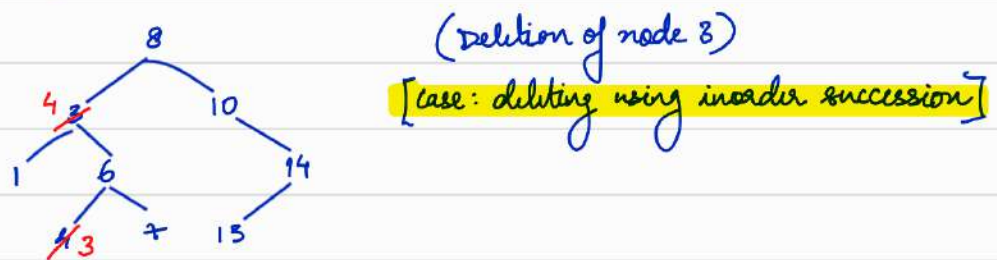
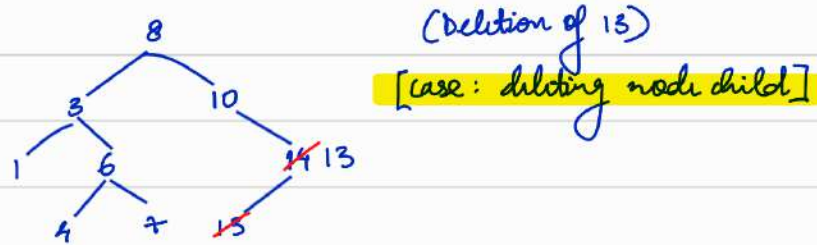
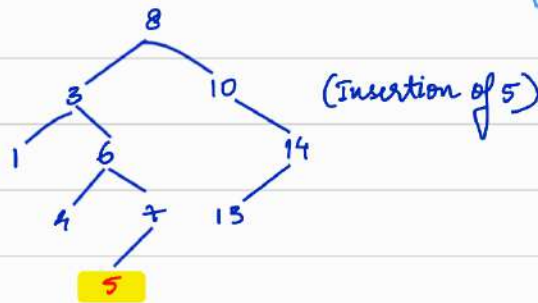
most efficient : $O(n \log n)$
→ $T_{\text{best}}(n) + T_{\text{search}}(n)$

⇒ $O(n \log n) + O(\log n)$

⇒ $O(n \log n)$

→ Introduction to BST (Insertion, Deletion, Search)

→ Deletion : inorder successor, leaf node (child),



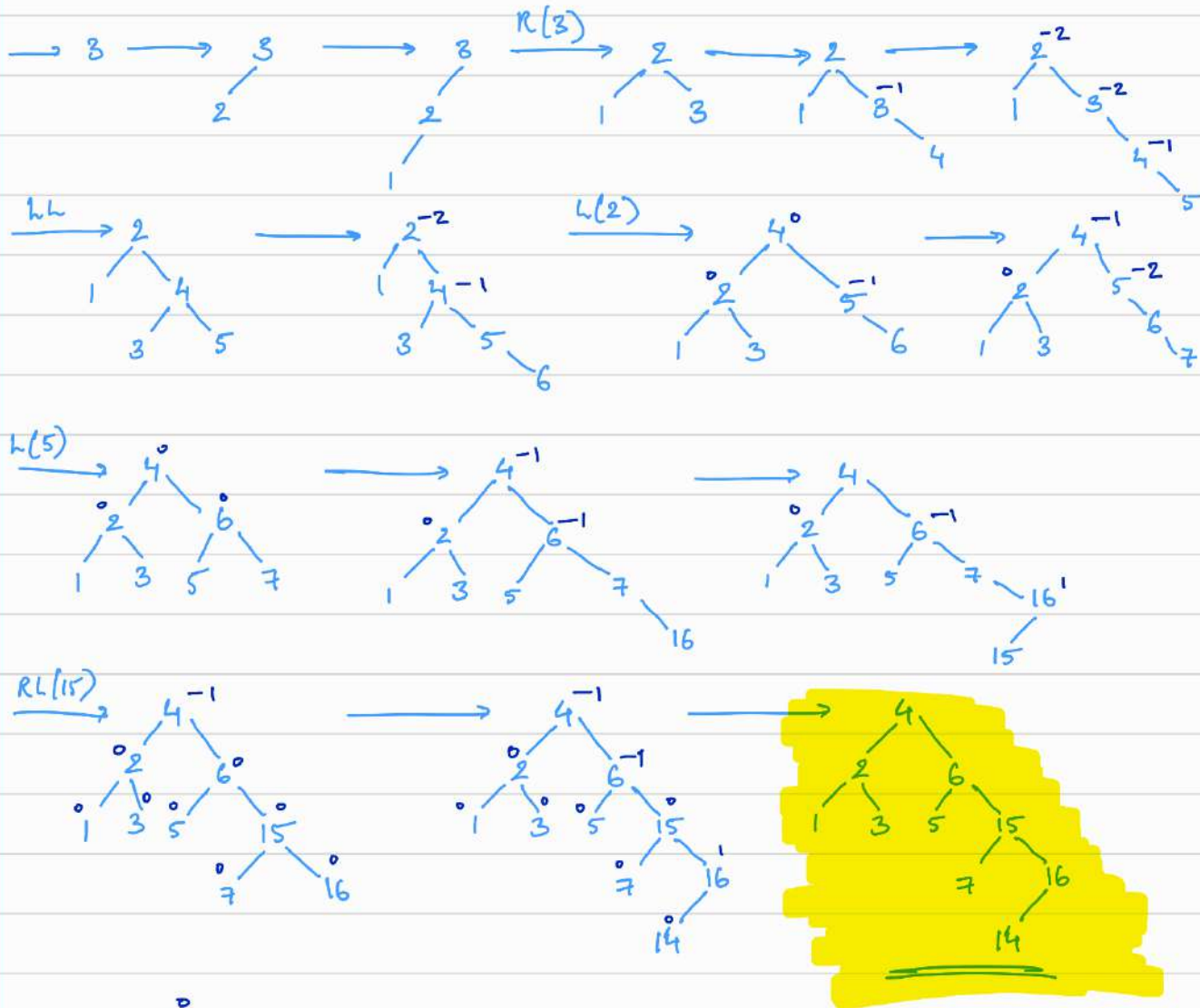
Delete 11, 7, 7, 9, 10, 15
insert 20

12/2/26

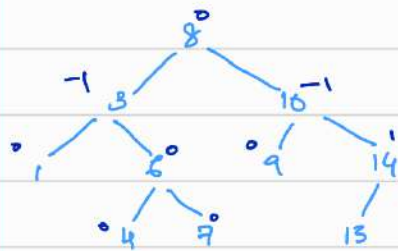
AVL trees

min. no. of nodes : $s(h)$
 $h \rightarrow$ height of the tree

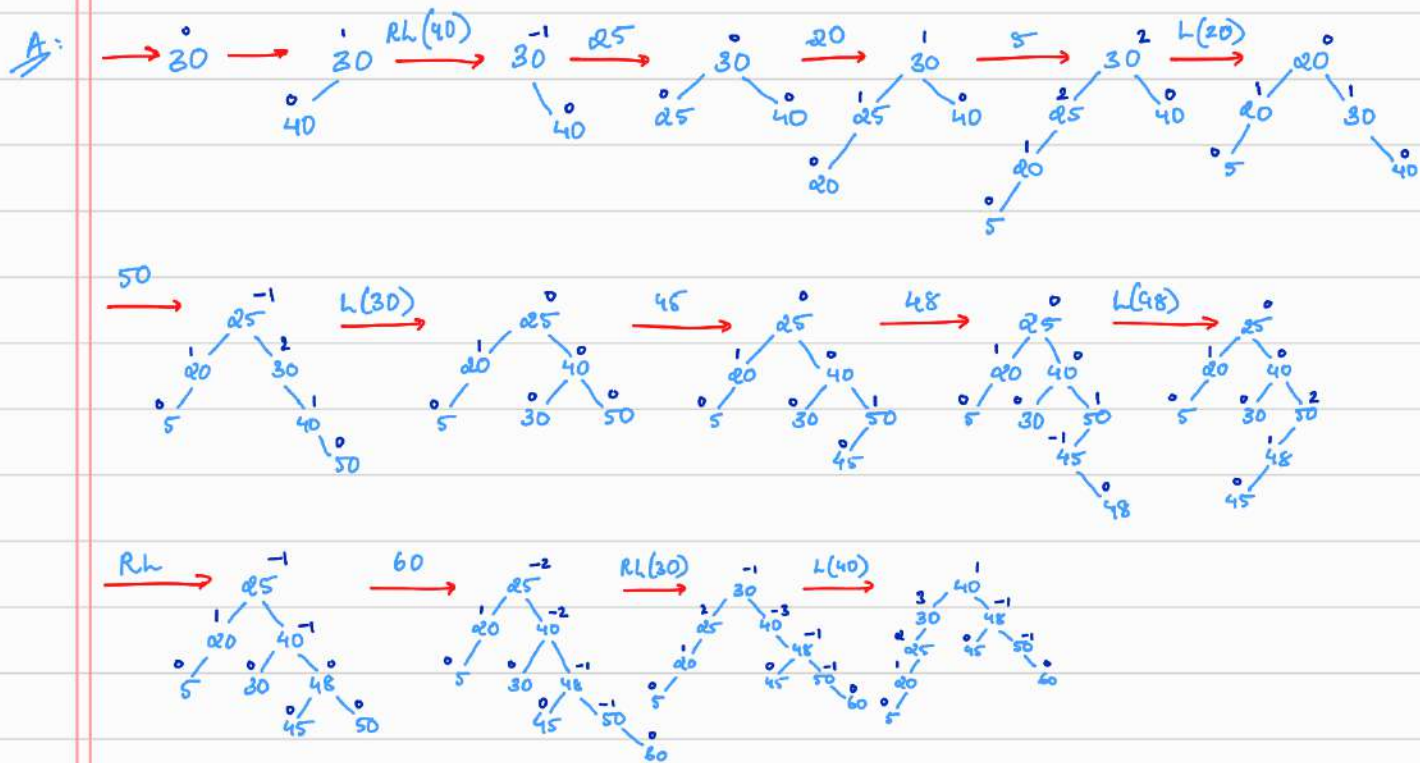
Q) 3, 2, 1, 4, 5, 6, 7, 16, 15, 14



Q)



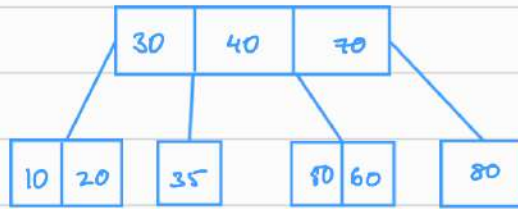
Q) Create AVL tree w. keys 30, 40, 25, 20, 5, 50, 45, 48, 60, 70, 65, 80, 90, 100, 10, 2
then delete the keys 60, 50, 45, 30



14/2/26

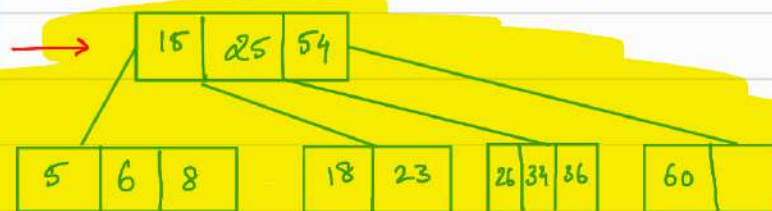
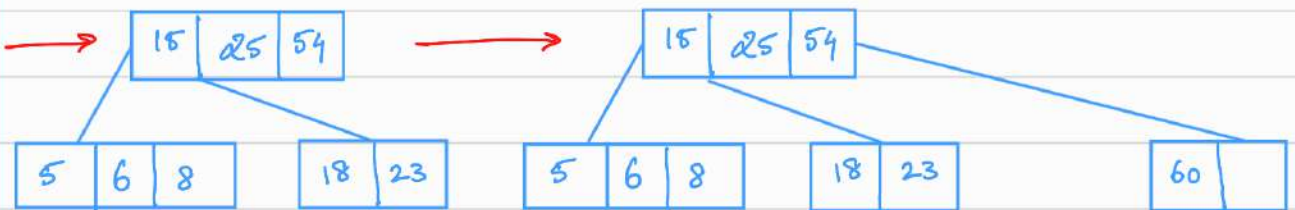
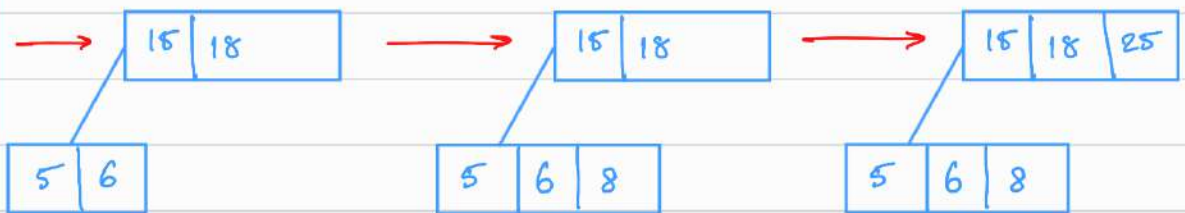
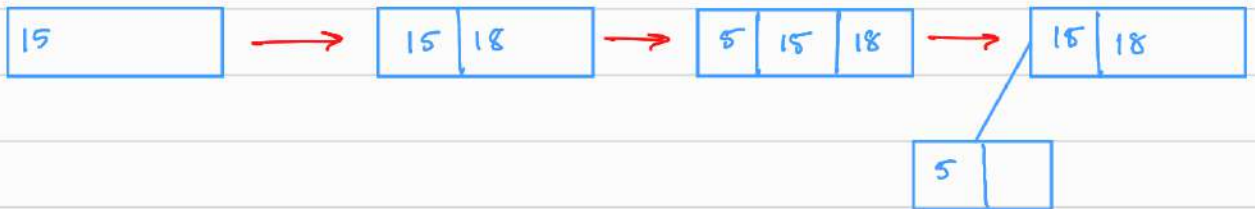
2, 3 Trees

Q) 10, 20, 40, 50, 60, 70, 80, 30, 35



Q) 15, 18, 5, 6, 8, 25, 23, 54, 60, 36, 34, 26

A:



① $\left\lceil \frac{m}{2} \right\rceil$

② Bottom-up

③ $m-1$ keys

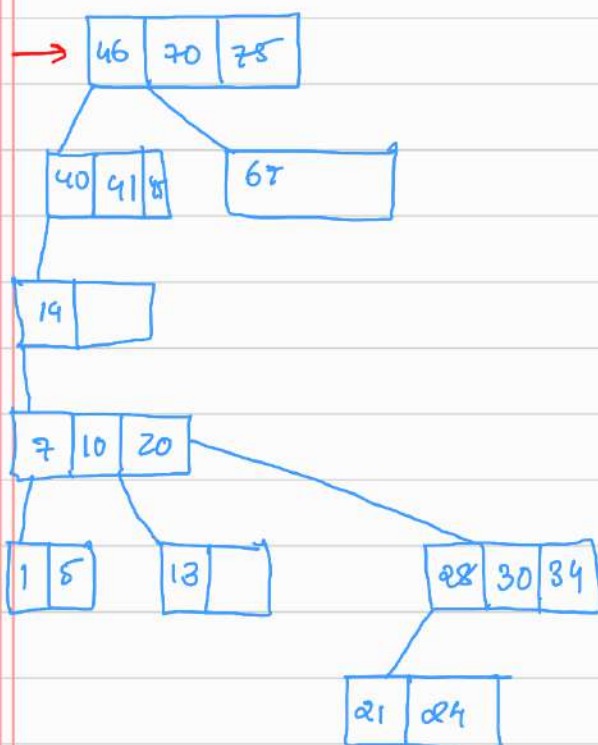
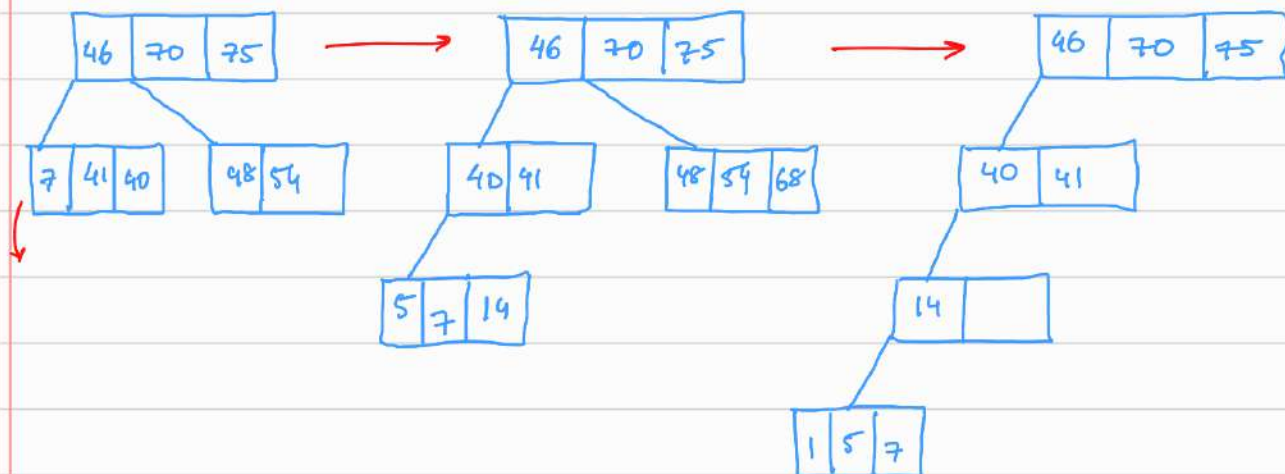
Q)

70, 75, 46, 7, 41, 54, 40, 48, 5, 14, 68, 1,
10, 20, 13, 21, 28, 24, 67, 30, 34, 45

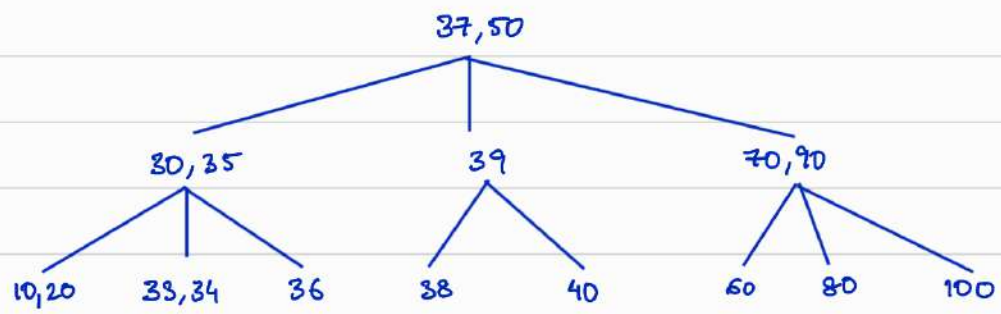
A:



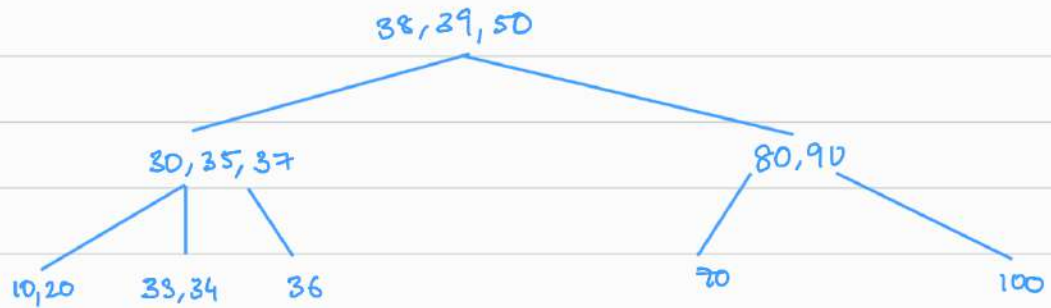
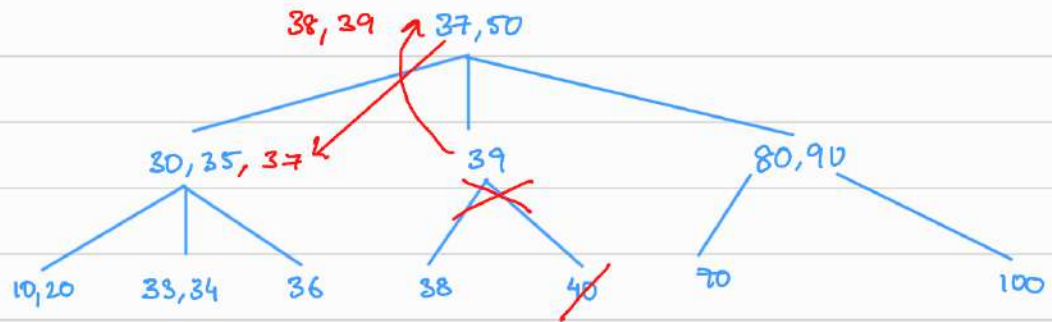
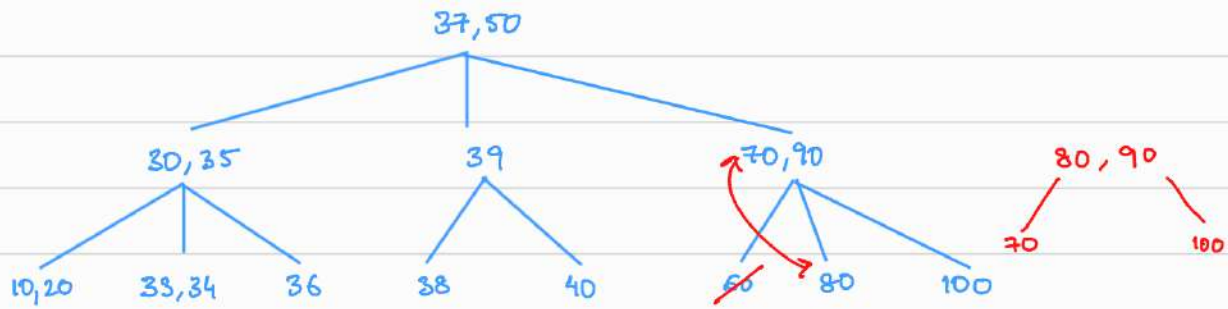
17th Feb
5.10



2)



A:



17/2/26

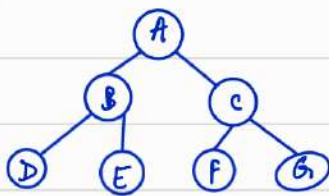


fig:- complete graph
+ full graph

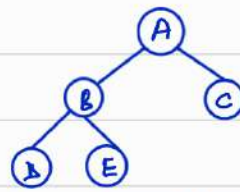


fig:- complete graph
only

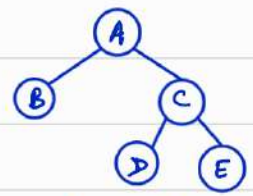


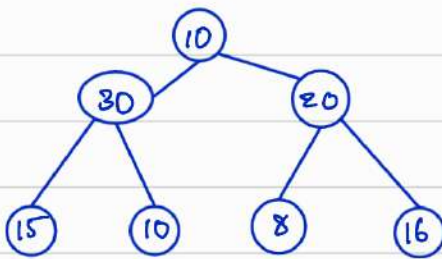
fig:- Not complete
nor full graph

→

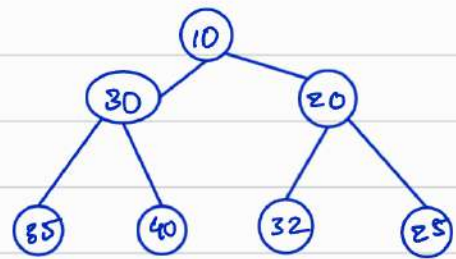
Heap

→ Max-Heap

→ Min-Heap

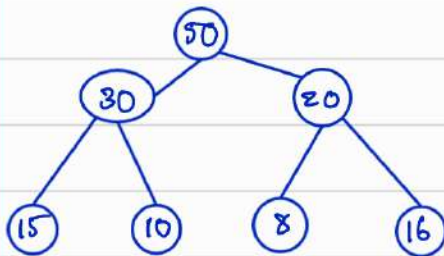


- parent $\geq$ child nodes
- then $\Rightarrow$ Max-Heap
- Duplicates are allowed.
- All descendants should be $\leq$ to their respective parent nodes.



- parent $\leq$ child nodes
- then $\Rightarrow$ Min-Heap
- Descendants should be $\geq$ to their respective parent nodes.
- Duplicates are not allowed

Q



0 1 2 3 4 5 6

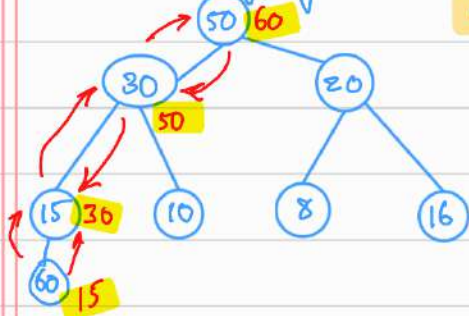
50 20 20 15 10 8 16

→ Verified as complete graph = heap

Insert 60 to the above heap

A:

insert to left of 15



0 1 2 3 4 5 6 7

50 20 20 15 10 8 16 60

$i = 7 \Rightarrow \left\lfloor \frac{7}{2} \right\rfloor \Rightarrow \left\lfloor \frac{7}{2} \right\rfloor \Rightarrow \left\lfloor 3.5 \right\rfloor \Rightarrow 3$ (swap w. 3)

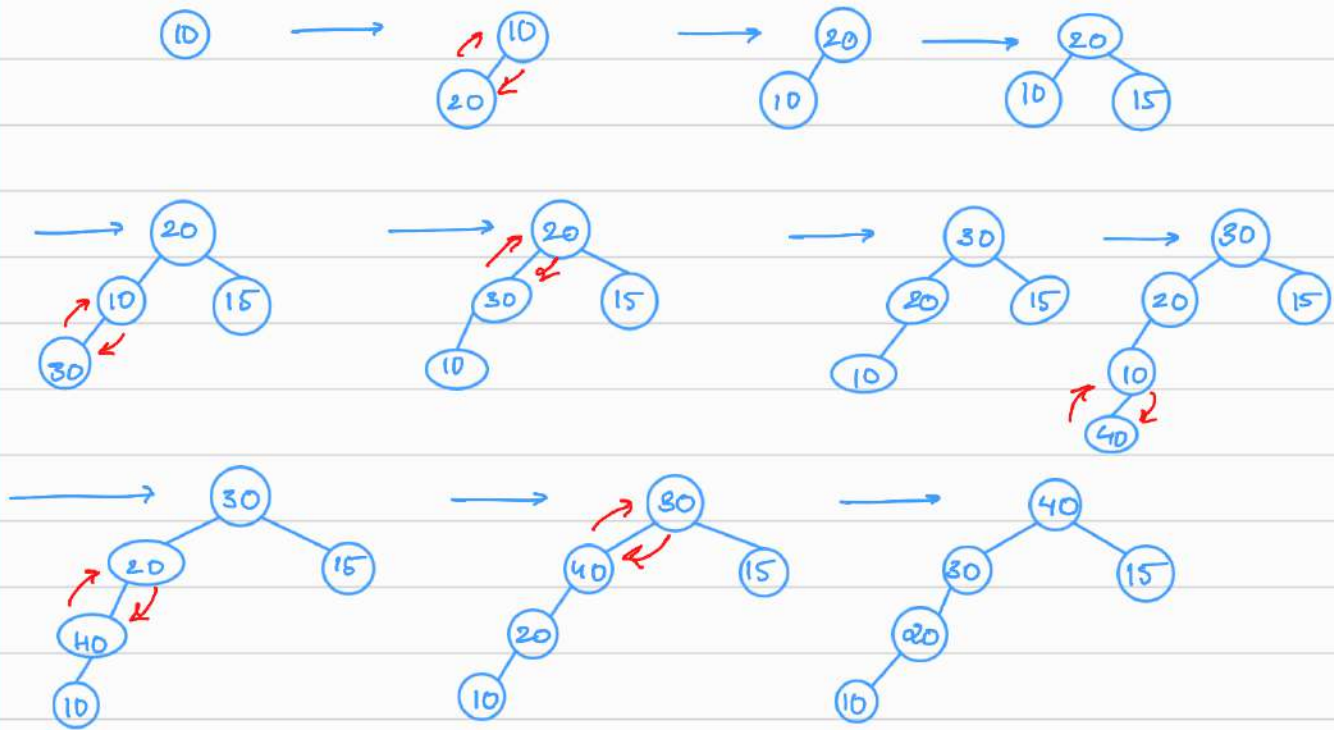
:

$i = 1 \Rightarrow \left\lfloor \frac{1}{2} \right\rfloor \Rightarrow \left\lfloor 0.5 \right\rfloor \Rightarrow 0$ (swap w. 0)

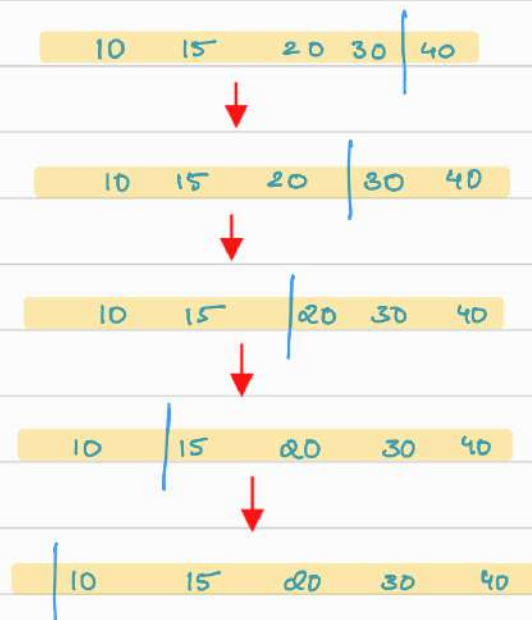
∴ Now it's a Max-Heap

Q) Create a **Max. Heap** inserting 10, 20, 15, 30, 40. Show step by step procedure

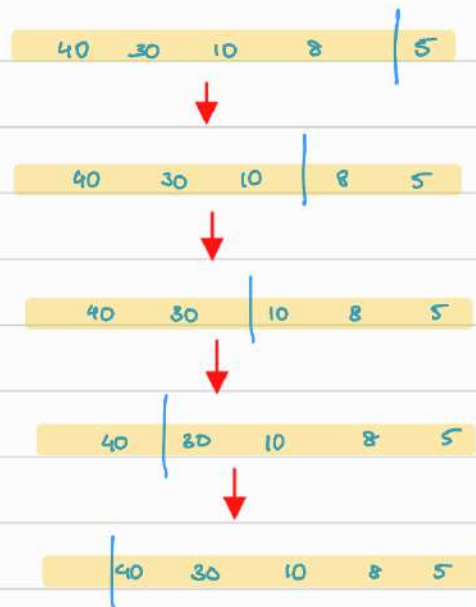
Ans:



→ Now, Deleting...



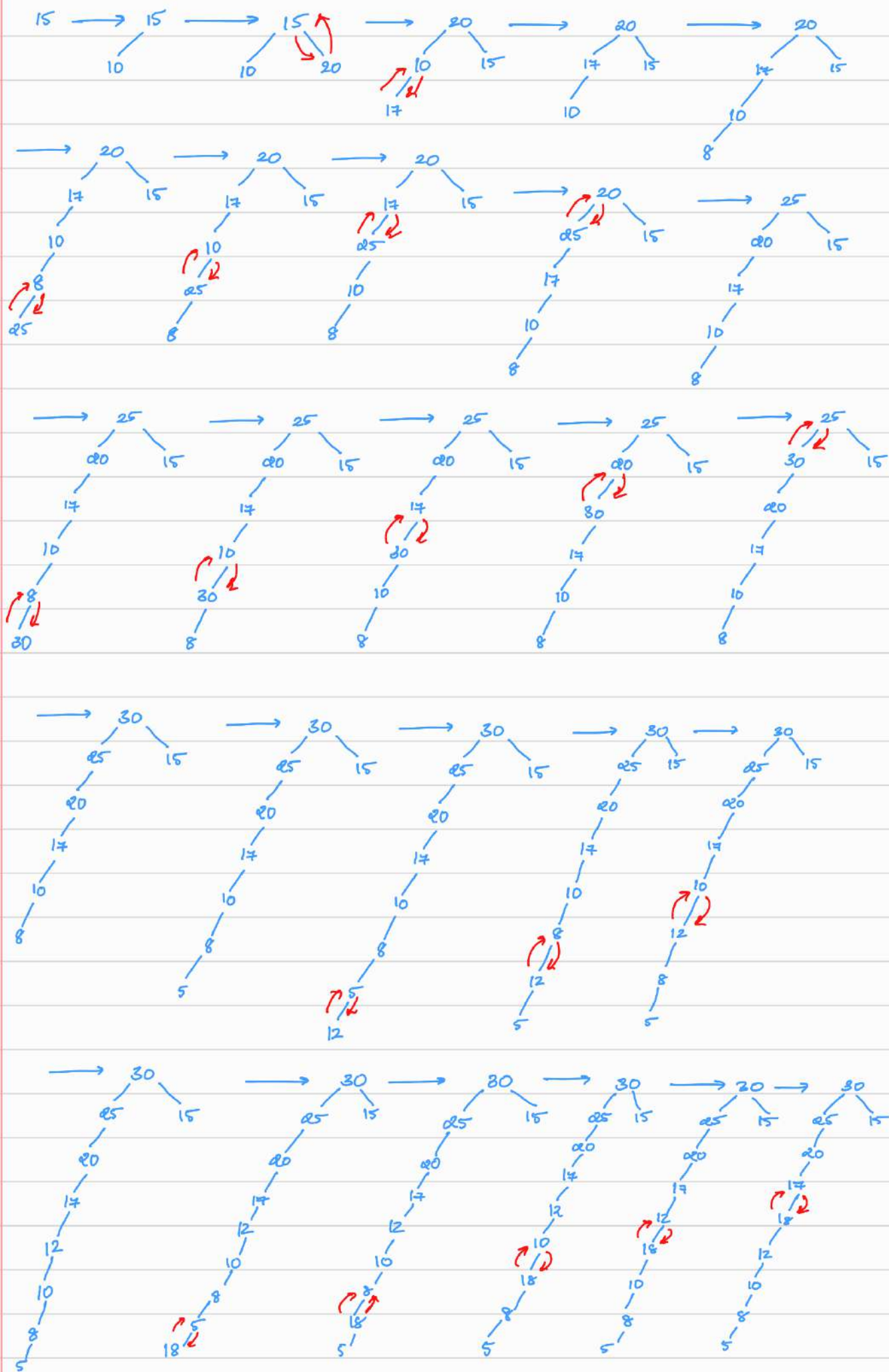
Create a **Min Heap** inserting 30, 10, 40, 5, 8. Show step by step procedure

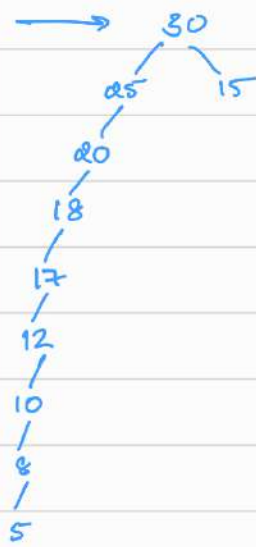


18/2/26

Q)

15, 10, 20, 17, 8, 25, 30, 5, 12, 18 $\Rightarrow$ Max. Heap





Now, Deleting....

5 8 10 12 15 17 18 20 25 | 30

5 8 10 12 15 17 18 20 | 25 30

5 8 10 12 15 17 18 | 20 25 30

5 8 10 12 15 17 | 18 20 25 30

5 8 10 12 15 | 17 18 20 25 30

5 8 10 12 | 15 17 18 20 25 30

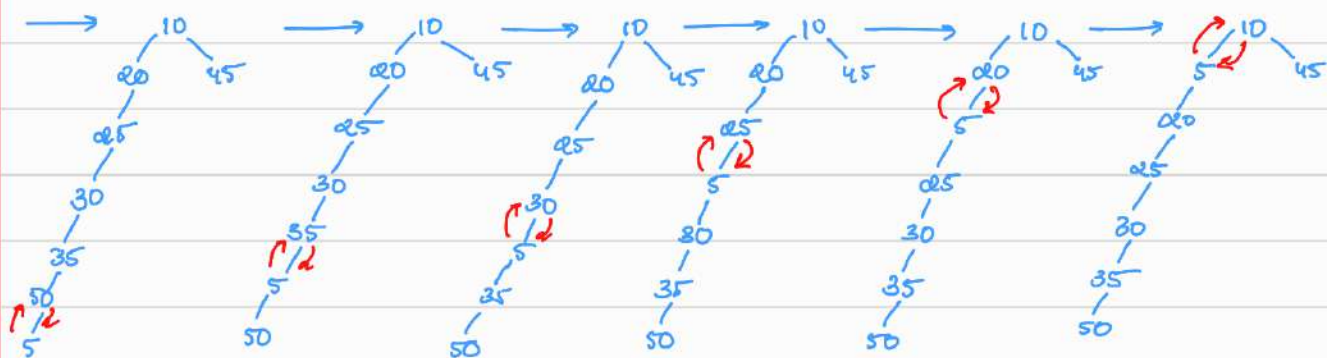
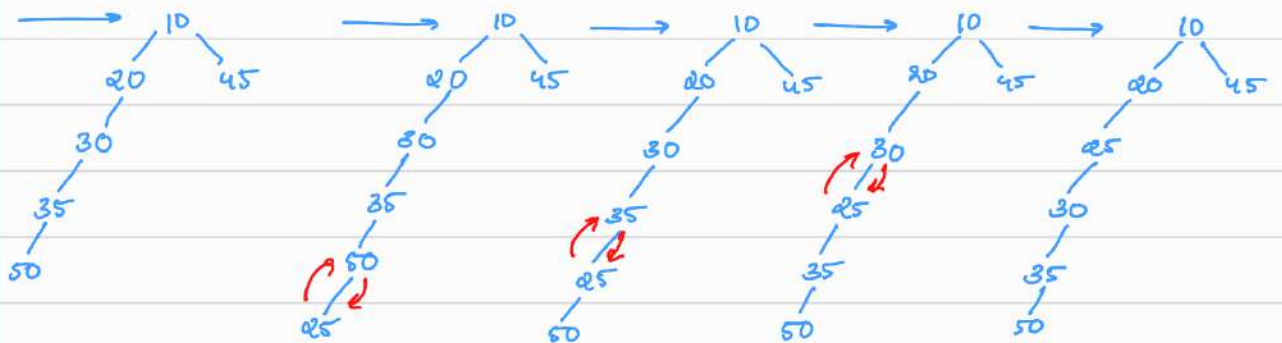
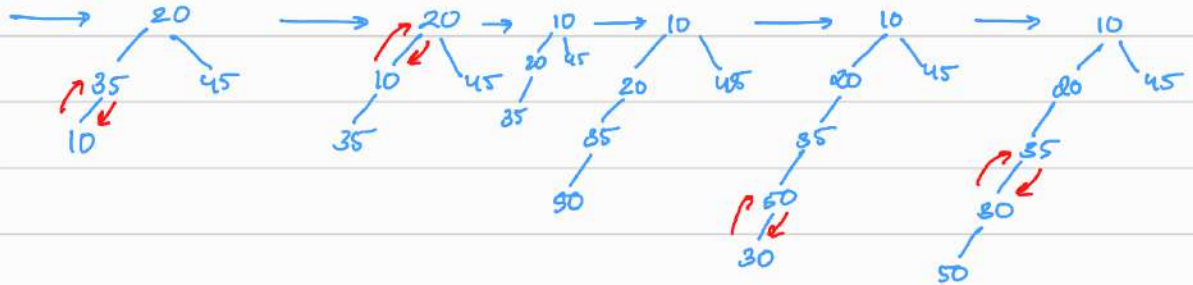
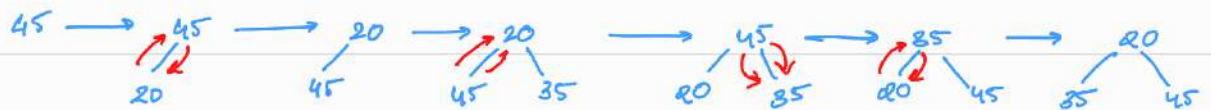
5 8 10 | 12 15 17 18 20 25 30

5 8 | 10 12 15 17 18 20 25 30

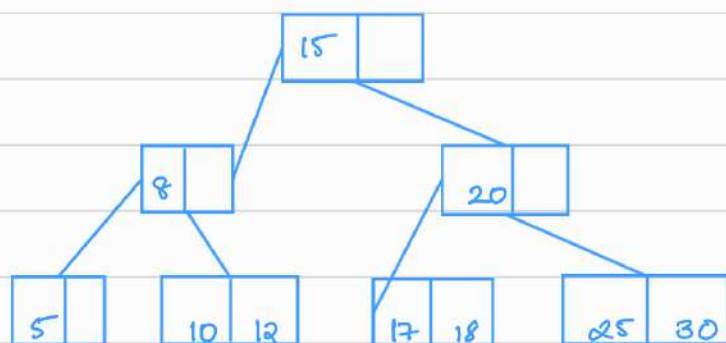
5 | 8 10 12 15 17 18 20 25 30

| 5 8 10 12 15 17 18 20 25 30

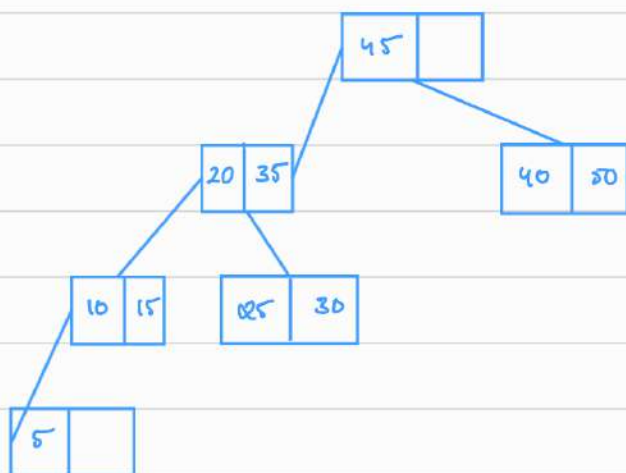
a) 45, 20, 35, 10, 50, 30, 25, 5, 40, 15 $\Rightarrow$ Min. Heap



Q) 15, 10, 20, 17, 8, 25, 30, 5, 12, 18 $\Rightarrow$ Construct a 2-3 Tree



Q) 45, 20, 35, 10, 50, 30, 25, 5, 40, 15 $\Rightarrow$ Construct a 2-3 Tree



19/2/26



Computing least Common Multiple

$$\text{lcm}(24, 60) = 120$$

$$\text{gcd}(24, 60) =$$

$$\text{lcm}(11, 5) = 55$$

$$\text{gcd}(11, 5) = 1$$

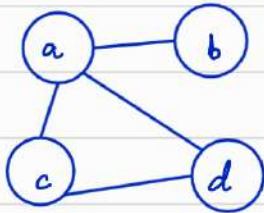
2	24, 60	
3	12, 30	
2	4, 10	
5	2, 5	
2		1
		1

Q) $\gcd(150, 36)$

150	36
36	6
6	0

$\gcd(b, a \bmod b)$

Q) Write the adjacency Matrix for



A:

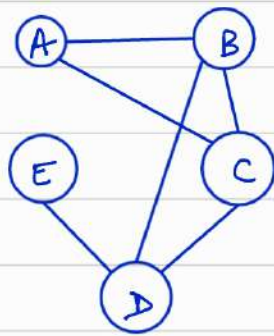
$$\begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \Rightarrow \text{Adj. Matrix}
 \end{matrix}$$

Considering Adj. Matrix = A, Now A^2 ,

$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$\begin{array}{lcl}
 a-b \times -0 & & \\
 a-c-d-a & 1 & \\
 a-d-c-a & 1 &
 \end{array}
 \left. \vphantom{\begin{array}{l} a-b \times -0 \\ a-c-d-a \\ a-d-c-a \end{array}} \right\} \text{other paths}$$

Q)



A:

$$A = \begin{matrix} & \begin{matrix} a & b & c & d & e \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$= \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 1 & 1 & 2 & 0 \\ 1 & 3 & 2 & 1 & 1 \\ 1 & 2 & 3 & 1 & 1 \\ 2 & 1 & 1 & 3 & 0 \\ 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

→ Reduction of Maximization/Minimization problem

$$\min f(x) = \max [-f(x)]$$

Eg:- $5+6=11$ [here 5 & 6 are max. values]
 $-5-6=-11$ [here -5 & -6 are min. values converted to get max. o/p]
 $=11$

Similarly, $\max f(x) = -\min [-f(x)]$

24/2/20
→

linear programming

Q) Stock, S $\rightarrow x_1$; Cash, C $\rightarrow x_2$; Bonds, B $\rightarrow x_3$
 $x_1 + x_2 + x_3 = 100$ ——— ①

A:

$$Z = 0.10x_1 + 0.07x_2 + 0.08x_3$$

$$x_1 \leq \frac{1}{3}x_2$$

[1st constraint]

$$x_3 = 0.25(x_1 + x_2) ; \quad x_1 = \frac{x_2}{3}$$

$$= 0.25\left(\frac{x_2}{3} + x_2\right)$$

$$= \frac{1}{4} \left[\frac{4x_2}{3} \right]$$

$$\frac{1}{3} + 1 = \frac{4}{3}$$

$$x_3 = \frac{1}{3}x_2$$

now substituting in ①,

$$x_1 + x_2 + x_3 = 100$$

$$\frac{x_2}{3} + x_2 + \frac{x_2}{3} = 100$$

$$\frac{x_2 + 3x_2 + x_2}{3} = 100$$

$$\frac{5x_2}{3} = 100$$

$$x_2 = \frac{100 \times 3}{5}$$

$$x_2 = 60$$

$$\text{Now, } x_1 = \frac{60}{3} = \underline{\underline{20}} \quad \text{and} \quad x_3 = \frac{60}{3} = \underline{\underline{20}}$$

$$\text{Substituting } x_1, x_2, x_3 \text{ in ①, } \Rightarrow 0.10(20) + 0.07(60) + 0.08(20) \\ \Rightarrow \underline{\underline{6.80}} \text{ profit}$$

→ Reduction to Graph Problems

→ Using state space graph :-

- ① States
- ② Transitions
- ③ initial state
- ④ Goal state



MIDTERM PORTION :- Till AVL Trees [Heap & 2, 3 Trees not included]

$2 \times 3, 4 \times 1, 3 \times 5$, MCQs $\rightarrow$ 5Q (1M)

$= 6 + 4 + 15 + 5 = 30$ marks

26/2/26

Tutorial Class

- Q) Write a C function for max. & minimum element in an array and find its space & time complexity as well. [Only function code need to be written]

A:

```
void maxele (int *arr[], int n, int *max, int *min) {
    *max = arr[0]
    *min = arr[0]
```

time complexity

 $\Rightarrow O(n)$

space complexity

 $\Rightarrow O(1)$

$\left\{ \begin{array}{l} n \\ n \\ n \end{array} \right.$

```
for (int i=0; i<n; i++) {
    if (arr[i] > *max) {
        *max = arr[i];
    }
    if (arr[i] < *min) {
        *min = arr[i];
    }
}
```

- Q) Show that $p(n)$ is asymptotically bigger than $q(n)$.

$$p(n) = 17n^2 2^n$$

$$q(n) = 100n 2^n + 33n$$

$$\lim_{n \rightarrow \infty} \frac{p(n)}{q(n)} = \frac{17n^2 2^n}{100n 2^n + 33n} = \frac{\cancel{n^2 2^n} (17)}{\cancel{n^2 2^n} \left[100 n^{-1} + \frac{33}{n 2^n} \right]}$$

$$\lim_{n \rightarrow \infty} = \frac{17}{100 + 0} = \frac{17}{100} \neq 0$$

$$\therefore \underline{\underline{p(n) = O(q(n))}}$$

Q) show that,

$$5n^3 + 6n^3 \neq \Omega(n^3 \log n)$$

A:

$$f(n) = 5n^3 + 6n^3 = 11n^3$$

$$g(n) = n^3 \log n$$

$$\therefore f(n) \neq \Omega(g(n))$$

Q) what does fun() do in general? find its space complexity.

```
int fun(int x, int y) {
    if (y == 0) return 0;
    return (x + fun(x, y-1));
}
```

28/2/26

Q

Multiply the following two matrix using Strassen's Matrix Multiplication Method. Multiply 2×2 matrix conventionally.

$$\begin{bmatrix} 5 & 3 & 1 & 2 \\ 5 & 1 & 5 & 2 \\ 1 & 2 & 3 & 4 \\ 1 & 1 & 5 & 5 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 3 & 3 \\ 5 & 5 & 2 & 2 \\ 2 & 5 & 2 & 5 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

A:

$$\begin{array}{c|c} A_1 & A_2 \\ \hline \begin{bmatrix} 5 & 3 \\ 5 & 1 \end{bmatrix} & \begin{bmatrix} 1 & 2 \\ 5 & 2 \end{bmatrix} \\ \hline \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} & \begin{bmatrix} 3 & 4 \\ 5 & 5 \end{bmatrix} \\ \hline A_3 & A_4 \end{array}$$

$$\begin{array}{c|c} B_1 & B_2 \\ \hline \begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix} & \begin{bmatrix} 3 & 3 \\ 2 & 2 \end{bmatrix} \\ \hline \begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix} & \begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix} \\ \hline B_3 & B_4 \end{array}$$

$$\begin{aligned} D = A_1(B_1 - B_2) &= \begin{bmatrix} 5 & 3 \\ 5 & 1 \end{bmatrix} \left[\begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix} - \begin{bmatrix} 3 & 3 \\ 2 & 2 \end{bmatrix} \right] \\ &= \begin{bmatrix} -1 & -1 \\ -7 & -7 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} E = A_4(B_3 - B_1) &= \begin{bmatrix} 3 & 4 \\ 5 & 5 \end{bmatrix} \left[\begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix} \right] \\ &= \begin{bmatrix} -13 & -4 \\ -15 & 0 \end{bmatrix} \end{aligned}$$

$$F = (A_3 + A_4)B_1 = \left[\begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 3 & 4 \\ 5 & 5 \end{bmatrix} \right] \begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix} = \begin{bmatrix} 84 & 34 \\ 36 & 36 \end{bmatrix}$$

$$G = (A_1 + A_2)B_4 = \left[\begin{bmatrix} 5 & 3 \\ 5 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 5 & 2 \end{bmatrix} \right] \begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix}$$

$$= \underline{\underline{\begin{bmatrix} 17 & 35 \\ 23 & 53 \end{bmatrix}}}$$

$$H = (A_3 - A_1)(B_1 + B_2) = \left[\begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 5 & 3 \\ 5 & 1 \end{bmatrix} \right] \left[\begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix} + \begin{bmatrix} 3 & 3 \\ 2 & 2 \end{bmatrix} \right]$$

$$= \begin{bmatrix} 6 & 5 \\ 6 & 2 \end{bmatrix} \begin{bmatrix} 4 & 4 \\ 7 & 7 \end{bmatrix}$$

$$= \underline{\underline{\begin{bmatrix} 59 & 59 \\ 38 & 28 \end{bmatrix}}}$$

$$I = (A_2 - A_4)(B_1 + B_4) = \left[\begin{bmatrix} 1 & 2 \\ 5 & 2 \end{bmatrix} - \begin{bmatrix} 3 & 4 \\ 5 & 5 \end{bmatrix} \right] \left[\begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix} + \begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix} \right]$$

$$= \begin{bmatrix} -2 & -2 \\ 0 & -3 \end{bmatrix} \begin{bmatrix} 3 & 6 \\ 6 & 6 \end{bmatrix}$$

$$= \underline{\underline{\begin{bmatrix} -18 & -24 \\ -18 & -18 \end{bmatrix}}}$$

$$J = (A_1 + A_4)(B_1 + B_4) = \left[\begin{bmatrix} 5 & 3 \\ 5 & 1 \end{bmatrix} + \begin{bmatrix} 3 & 4 \\ 5 & 5 \end{bmatrix} \right] \left[\begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix} + \begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix} \right]$$

$$= \begin{bmatrix} 8 & 7 \\ 10 & 6 \end{bmatrix} \begin{bmatrix} 3 & 6 \\ 6 & 6 \end{bmatrix}$$

$$= \underline{\underline{\begin{bmatrix} 66 & 90 \\ 66 & 96 \end{bmatrix}}}$$

$$C_1 = E + I + J - G$$

$$= \begin{bmatrix} -13 & -4 \\ -15 & 0 \end{bmatrix} + \begin{bmatrix} -18 & -24 \\ -18 & -18 \end{bmatrix} + \begin{bmatrix} 66 & 90 \\ 66 & 96 \end{bmatrix} - \begin{bmatrix} 17 & 35 \\ 23 & 53 \end{bmatrix}$$

$$= \begin{bmatrix} 85 & 62 \\ 33 & 78 \end{bmatrix} - \begin{bmatrix} 17 & 35 \\ 23 & 53 \end{bmatrix} = \underline{\underline{\begin{bmatrix} 18 & 27 \\ 10 & 25 \end{bmatrix}}}$$

$$C_2 = D + G = \begin{bmatrix} -1 & -1 \\ -7 & -7 \end{bmatrix} + \begin{bmatrix} 17 & 35 \\ 23 & 53 \end{bmatrix}$$

$$= \underline{\underline{\begin{bmatrix} 16 & 34 \\ 16 & 46 \end{bmatrix}}}$$

$$C_3 = E + F = \begin{bmatrix} -13 & -4 \\ -15 & 0 \end{bmatrix} + \begin{bmatrix} 34 & 34 \\ 36 & 36 \end{bmatrix} = \underline{\underline{\begin{bmatrix} 21 & 30 \\ 21 & 36 \end{bmatrix}}}$$

$$C_4 = D + H + J - F$$

$$= \begin{bmatrix} -1 & -1 \\ -7 & -7 \end{bmatrix} + \begin{bmatrix} 59 & 59 \\ 38 & 38 \end{bmatrix} + \begin{bmatrix} 66 & 90 \\ 66 & 96 \end{bmatrix} - \begin{bmatrix} 21 & 30 \\ 21 & 36 \end{bmatrix}$$

$$= \begin{bmatrix} 124 & 148 \\ 97 & 127 \end{bmatrix} - \begin{bmatrix} 21 & 30 \\ 21 & 36 \end{bmatrix}$$

$$= \underline{\underline{\begin{bmatrix} 103 & 118 \\ 76 & 91 \end{bmatrix}}}$$

$$\therefore C = \underline{\underline{\begin{bmatrix} 18 & 27 & 16 & 34 \\ 10 & 25 & 16 & 46 \\ 21 & 30 & 103 & 118 \\ 21 & 36 & 76 & 91 \end{bmatrix}}}$$

Q) Solve the following using Master's theorem

$$T(n) = \overset{\rightarrow 2^3}{\underbrace{128}_a} T\left(\underset{\substack{\uparrow \\ b}}{\frac{n}{2}}\right) + 6n^{\overset{\rightarrow 2^3}{8}}, \text{ where } n \geq 2 \text{ \& power of } 2$$

A:

$$\text{If } a < b^d \Rightarrow T(n) \in O(n^2)$$

$$\text{If } a = b^d \Rightarrow T(n) \in O(n^d \log n)$$

$$\text{If } a > b^d \Rightarrow T(n) \in O(n^{\log_b a})$$

$$a = 128; b = 2; d = 2$$

$$\Rightarrow a ? b^d$$

$$\Rightarrow 128 ? 2^2$$

$$\Rightarrow 128 > 2^2$$

$\therefore T(n) \in O(n^{\log_2 128})$ by Master's theorem
by substitution we get,

$$\Rightarrow O(n^{\log_2 128})$$

$$\Rightarrow \underline{\underline{O(n^7)}}$$

14/3/26

→

Hashing

→ Goal: perform inserts, deletes & finds in **constant avg. time**

→ Topics: Hash table, hash function, collisions

→ Collision handling:

Open hashing: → **separate chaining**

Close hashing: → Open addr.: **linear probing, quadratic prob., double hashing**

→ Hash table ⇒ a table of values/keys.

Hash func. ⇒ map the keys in hash table

collisions ⇒

Ex:

15, 55, 60, 10, 5, 86 ; n=6

Search: 60

Sol:

↓ ↓ ↓ ↓ ↓ ↓
3 1 0 4 5 0 ⇒ key values

Ex:

n=15 → 15, 55, 60, 10, 5, 86

Sol:

↓ ↓ ↓ ↓ ↓ ↓
0 10 5 10 5 86 ⇒ key values

Ex:

n=15 ; $f(x) = x \div 15$

$f(x) = 25$	129	35	2501	47	36
10	9	5	11	2	6

Sol:

searching 60? ⇒ $60 \div 15 \Rightarrow 4 \times 15 \Rightarrow 60 - 60 \Rightarrow \underline{0}$

∴ No collision

Ex:

Same as above but Search=65.

Sol:

searching 65 ⇒ $65 \div 15 \Rightarrow 65 - 60 \Rightarrow \underline{5}$

∴ collision occurs @ 5 w. 35 & 65.

Ex:

29, 16, 14, 99, 127 ⇒ Sep. Chaining (Open Hashing) to above q.

Sol:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
		↓			↓	↓			↓	↓	↓			
		47			35	36			129	25	2501			

Handling Collisions : linear Probing [close Hashing]

Ex:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
		↓			↓	↓			↓	↓	↓			
		47			35	36			129	25	2501			

for 29, 16, 14, 99, 127?

Sol:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

16 47 14 35 36 127 129 25 2501 99 29

$x \div 15$ $t+4$ $x \div 15$ $t+3$ $x \div 15$

→ Handling Collisions: Double Hashing [closed Hashing]

for $f_1(x) = t$; $f_2(x) = d$

$(t+d) \% N, (t+2d) \% N, (t+3d) \% N, \dots$ etc.

where $f_2(x) = R - (x \% R)$; where $R \rightarrow$ prime no. of N

Ex:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	↓	↓	↓		↓	↓	↓		↓	↓	↓	↓		↓
	16	47	14		35	36	127		129	25	2501	99		29

Sol:

$$f_2(14) = d = 8$$

$$; N=15, R=11, d=8$$

$$f_2(14) = 11 - (14 \% 11)$$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	↓	↓	↓		↓	↓	↓		↓	↓	↓	↓		↓
	16	47	14		35	36	127		129	25	2501	99		29

$(t+8d) \% N$ $\rightarrow (t+3d) \% N$ for $d=$

12/6/26

→

Re-Hashing

$m \rightarrow$ no. of keys in hash table

If $m/n \leq 0.75 \Rightarrow$ size of hash table, $n = 2n$

then rehash all the keys from the beginning
w. the new hash table

Else insert in same hash table

→

Input Enhancement in string Matching

- m character string $\rightarrow$ pattern
- long string $\rightarrow$ text
- 2 sub-algorithms $\Rightarrow$
 - Horspool's
 - Boyer



Horspool's Algorithm

Eg:- Text : C O M P U T E R
 pattern : P U T

M ≠ T
 Take 8 shifts
 (as per pattern size)

T = T, U = U, P = P
 pattern matched!

} Brute force method

P U T

Ex:-

A	B	C	D	E	F	G	H	I	J		Z	—
6	6	6	6	6	6	6	6	6	6		6	

text: J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
 pattern: B A R B E R

A:-

A	B	C	D	E	F	G	H	...	R		Z	—
6 ₄	6 ₂	6	6	6 ₁	6	6	6	6	6 ₃	6	6	

↳ using shift table

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
 B A R B E R

shift 4

B A R B E R

shift 1

B A R B E R

shift 6

B A R B E R

shift 2

B A R B E R

shift 3

B A R B E R

∴ pattern matched after 5 shifts

Ex:

A	B	C	D	E	F	G	H	I	J	...	Z	—
8	8	8	8	8	8	8	8	8	8	8	8	8

text: COMPUTER — SCIENCE — ENGINEERING
 pattern: ENGINEER

A:

A	B	C	...	E	...	G	...	I	...	N	...	R	...	Z	—
8	8	8	8	8 ₁	8	8 ₅	8	8 ₄	8	8 ₃	8	8	8	8	8

COMPUTER — SCIENCE — ENGINEERING
 ENGINEER
 ENGINEER
 ENGINEER
 ENGINEER

shifts
 shift 1
 shift 8

∴ pattern matched after 4 shifts

Ex:

text: KANNADA — FILM — INDUSTRY — ICON — IS — RAJKUMAR
 pattern: RAJKUMAR

A:

A	B	C	...	I	K	...	M	...	R	...	U	...	Z	—
1 ₈	8	8	8	5 ₈	4 ₈	8	2 ₈	8	7 ₈	8	3 ₈	8	8	8

KANNADA — FILM — INDUSTRY — ICON — IS — RAJKUMAR
 RAJKUMAR
 RAJKUMAR
 RAJKUMAR
 RAJKUMAR
 RAJKUMAR
 RAJKUMAR
 RAJKUMAR

shift 1
 shift 8
 shift 8
 shift 8
 shift 1
 shift 5

∴ pattern matched after 6 shifts

⇒ Boyer Moore Algorithm

Ex: text: BESS_KNEW_ABOUT_BAOBABBS
pattern: BAOBAB

A: good suffix table :-

k	pattern	d ₂
1	B A D <u>B A B</u>	2
2	<u>B</u> A D ^x <u>B A B</u>	5
3	<u>B</u> A D <u>B A B</u>	5
4	<u>B</u> A D <u>B A B</u>	5
5	<u>B</u> A D <u>B A B</u>	5

Hash Table :-

A	B	c	...	O	...	z	—
6 ₁	6 ₂	6	6	6 ₃	6	6	6

BESS_KNEW_ABOUT_BAOBABBS
BAOBAB

Ex:

text: STOU__TOR_STOR

pattern: STOR

#:

Good suffix table:-

k	pattern	d _k
1	STOR <u>R</u>	4
2	STOR <u>OR</u>	4
3	STOR <u>TOR</u>	4

A	B	C		D	...	R	S	T		Z	—
4	4	4	4	1	4	4	3	2	4	4	4

18/5/26
→

Dynamic Programming

→ 0/1 Knapsack

(Weight, Profit, Capacity)

Set Method

tabular Method

Ex:

$$m = 8$$

$$P = \{1, 2, 5, 6\}$$

$$n = 4$$

$$W = \{2, 3, 4, 5\}$$

Q:

tabular Method:-

		W									
			0	1	2	3	4	5	6	7	8
P	W	0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

Now Backtracking Method:-

		W									
			0	1	2	3	4	5	6	7	8
P	W	0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

		P	W
x_1	0	1	2
x_2	1	2	3
x_3	0	5	4
x_4	1	6	5

for $x=8 \Rightarrow \{6, 2\}$ for $W=5 > 3\}$

$$\Rightarrow 8 - 6 = 2$$

for $x=2 \Rightarrow \{3, 3\}$ but it has appeared before

$$\Rightarrow \{2, 3\} \Rightarrow 2 - 2 = 0$$

for $x=0 \Rightarrow \{1, 0\}$ appeared before

$$\Rightarrow \{0, 0\} \Rightarrow 0 - 0 = 0$$

Applying formula (to verify) :-

$$v[i, w] = \max \{ v[i-1, w], v[i-1, w-w[i]] + p[i] \}$$

$$\begin{aligned} v[4, 7] &= \max \{ v[4-1, 7], v[4-1, 7-w[4]] + p[4] \} \\ &= \max \{ v[3, 7], v[3, 0] + 6 \} \\ &= \end{aligned}$$

Set Method :-

$$\begin{aligned} s^0 &= \{(0, 0)\} \\ s_1^0 &= \{(1, 2)\} \quad (p, w) \end{aligned} \quad \begin{aligned} s^1 &= \{(0, 0), (1, 2)\} \\ s_1^1 &= \{(2, 3), (3, 5)\} \end{aligned} \quad \begin{aligned} s^2 &= \{(0, 0), (1, 2), (2, 3), (3, 5)\} \\ s_1^2 &= \{(5, 4), (6, 6), (7, 7), (8, 9)\} \end{aligned}$$

$$\rightarrow s^3 = \{(0, 0), (1, 2), (2, 3), \cancel{(3, 5)}, (5, 4), (6, 6), (7, 7)\}$$

$\begin{matrix} p & w \\ \cancel{3} & \cancel{5} \\ p < w \end{matrix}$

$$s^3 = \{(0, 0), (1, 2), (2, 3), (5, 4), (6, 6), (7, 7)\}$$

$$s_1^3 = \{(6, 5), (7, 7), (8, 8), \cancel{(11, 1)}, \cancel{(12, 11)}, \cancel{(13, 12)}\}$$

$$s_1^4 = \{(6, 5), (7, 7), (8, 8)\}$$

$$\rightarrow s^4 = \{(0, 0), (1, 2), (2, 3), (5, 4), (6, 5), \cancel{(6, 6)}, (7, 7), (8, 8)\}$$

88

65

23
23
00

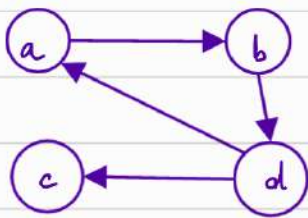
★ Fibonacci No.'s $\Rightarrow O(2^n)$ [for Dynamic Programming]

$\Rightarrow$ reducing same task

9/3/26

→ Marshall's Transitive Closure (DP: w/o weights)

Ex:



A:

Constructing adj. matrix for the above Directed graph,

$$= A = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R^0 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$b \rightarrow d$ [no change as 1]
 $b \rightarrow a$ & $a \rightarrow c = 0$
 $d \rightarrow a$ & $a \rightarrow b = 1$

$$R^1 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

$a \rightarrow d$: $a \rightarrow b$ & $b \rightarrow d = 1$
 $d \rightarrow d$: $b \rightarrow d$ & $d \rightarrow b = 1$

$$R^2 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

[no changes, all zeros]

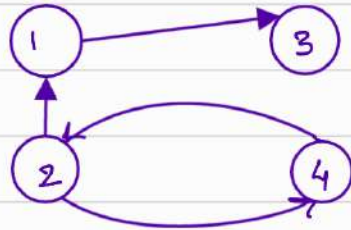
$$R^3 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$a \rightarrow a$: $a \rightarrow d$ & $d \rightarrow a = 1$
 $a \rightarrow c$: $d \rightarrow c$ & $a \rightarrow d = 1$
 $b \rightarrow a$: $d \rightarrow a$ & $b \rightarrow d = 1$
 $b \rightarrow b$: $d \rightarrow b$ & $b \rightarrow d = 1$
 $b \rightarrow c$: $d \rightarrow c$ & $b \rightarrow d = 1$

$$R^1 = \begin{matrix} & a & b & c & d \\ a & 1 & 1 & 1 & 1 \\ b & 1 & 1 & 1 & 1 \\ c & 0 & 0 & 0 & 0 \\ d & 1 & 1 & 1 & 1 \end{matrix}$$

Time complexity: $O(n^3)$

a)



$\Rightarrow$

$$\begin{matrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 & 1 \\ 3 & 0 & 0 & 0 & 0 \\ 4 & 0 & 1 & 0 & 0 \end{matrix}$$

A:

$$R^0 = \begin{matrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 & 1 \\ 3 & 0 & 0 & 0 & 0 \\ 4 & 0 & 1 & 0 & 0 \end{matrix}$$

$2 \rightarrow 3: 2 \rightarrow 1 \wedge 1 \rightarrow 3 = 1$

$$R^1 = \begin{matrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 0 & 1 & 0 \\ 2 & 1 & 0 & 1 & 1 \\ 3 & 0 & 0 & 0 & 0 \\ 4 & 0 & 1 & 0 & 0 \end{matrix}$$

$4 \rightarrow 1: 2 \rightarrow 1 \wedge 4 \rightarrow 2 = 1$

$4 \rightarrow 3: 4 \rightarrow 2 \wedge 2 \rightarrow 3 = 1$

$4 \rightarrow 4: 4 \rightarrow 2 \wedge 2 \rightarrow 4 = 1$

$$R^2 = \begin{matrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 0 & 1 & 0 \\ 2 & 1 & 0 & 1 & 1 \\ 3 & 0 & 0 & 0 & 0 \\ 4 & 1 & 1 & 1 & 1 \end{matrix}$$

$$R^3 = \begin{matrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 0 & 1 & 0 \\ 2 & 1 & 0 & 1 & 1 \\ 3 & 0 & 0 & 0 & 0 \\ 4 & 1 & 1 & 1 & 1 \end{matrix}$$

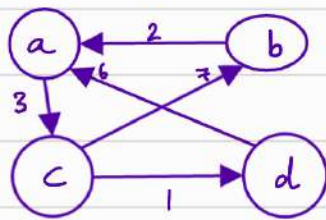
$2 \rightarrow 2: 4 \rightarrow 2 \wedge 2 \rightarrow 4 = 1$

$$R^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix} = T \text{ (Transitive closure)}$$

$\Downarrow$
final matrix

→ Floyd's Algorithm (DP: w. weights) / All Pairs shortest paths

Ex:



$$\Rightarrow \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$R^0 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$b \rightarrow c: 2 + 3 = 5 < \infty$$

$$d \rightarrow c: 6 + 3 = 9 < \infty$$

$$R^1 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{bmatrix} \end{matrix}$$

$$c \rightarrow a: 2 + 7 = 9 < \infty$$

$$c \rightarrow c: 7 + 5 = 12 > 0 \text{ x}$$

$$R^2 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{bmatrix} \end{matrix}$$

$$d \rightarrow a: 9 + 3 = 12 > 0 \text{ x}$$

$$a \rightarrow b: 7 + 3 = 10 < \infty$$

$$a \rightarrow d: 3 + 1 = 4 < \infty$$

$$b \rightarrow d: 5 + 1 = 6 < \infty$$

$$d \rightarrow b: 7 + 9 = 16 < \infty$$

$$R^3 = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 9 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{matrix}$$

$$c \rightarrow a: 6+1=7 < 9$$

$$R^4 = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{matrix}$$

Time Complexity: $O(n^3)$

20/8/26

Ex:

$$\Rightarrow \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

, solve using Warshall's Algorithm.

A:

$$R^0 = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

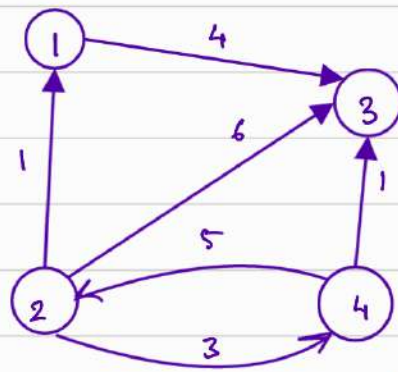
$$R^1 = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$R^2 = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$R^3 = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$R^4 = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Ex:



$$\Rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 6 & 3 \\ \infty & \infty & 0 & \infty \\ \infty & 5 & 1 & 0 \end{bmatrix} \end{matrix}$$

→ Solve using Floyd's Algorithm

$$R^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 6 & 3 \\ \infty & \infty & 0 & \infty \\ \infty & 5 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$2 \rightarrow 3: 4 + 1 = 5 < 6$$

$$R^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ \infty & 5 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$4 \rightarrow 1: 1 + 5 = 6 < \infty$$

$$4 \rightarrow 3: 5 + 5 = 10 > 1 \quad \times$$

$$R^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$2 \rightarrow 3: 3 + 1 = 4 < 5$$

$$R^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 4 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{bmatrix} \end{matrix}$$

Ex:

$$\begin{bmatrix} 0 & 2 & \infty & 1 & 8 \\ 6 & 0 & 3 & 2 & \infty \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & \infty & \infty & \infty & 0 \end{bmatrix}, \text{ using floyd}$$

A:

$$R^0 = \begin{bmatrix} 0 & 2 & \infty & 1 & 8 \\ 6 & 0 & 3 & 2 & \cancel{14}_{14} \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & \cancel{5} & \infty & \cancel{4} & 0 \end{bmatrix}$$

$$R^1 = \begin{bmatrix} 0 & 2 & \cancel{5} & 1 & 8 \\ 6 & 0 & 3 & 2 & 14 \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & 5 & \cancel{8} & 4 & 0 \end{bmatrix}$$

$$R^2 = \begin{bmatrix} 0 & 2 & \infty & 1 & 8 \\ 6 & 0 & 3 & 2 & 14 \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & 5 & 8 & 4 & 0 \end{bmatrix}$$

$$R^3 = \begin{bmatrix} 0 & 2 & \cancel{3} & 1 & \cancel{8}_4 \\ 6 & 0 & 3 & 2 & \cancel{14}_5 \\ \infty & \infty & 0 & 4 & \cancel{4}_4 \\ \infty & \infty & 2 & 0 & 3 \\ 3 & 5 & \cancel{6} & 4 & 0 \end{bmatrix}$$

$$R^5 = \begin{bmatrix} 0 & 2 & 3 & 1 & 4 \\ 6 & 0 & 3 & 2 & 5 \\ \cancel{10} & \cancel{12} & 0 & 4 & 7 \\ \cancel{6} & \cancel{8} & 2 & 0 & 3 \\ 3 & 5 & 6 & 4 & 0 \end{bmatrix}$$

$$R^6 = \begin{bmatrix} 0 & 2 & 3 & 1 & 4 \\ 6 & 0 & 3 & 2 & 5 \\ 10 & 12 & 0 & 4 & 7 \\ 6 & 8 & 2 & 0 & 3 \\ 3 & 5 & 6 & 4 & 0 \end{bmatrix}$$

*HW

NOTE:-

go thru Spanning Tree before Greedy Technique

24/3/26

→

Greedy Techniques

① Prim's Algo. [Always has to use MST]

② Kruskal's Algo. [Always has to use MST]

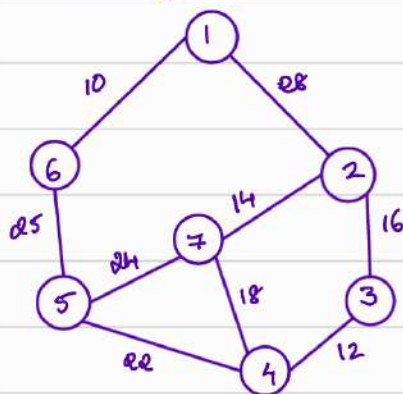
③ Huffman Coding

④ Dijkstra's

→

Prim's Algo.

Ex:

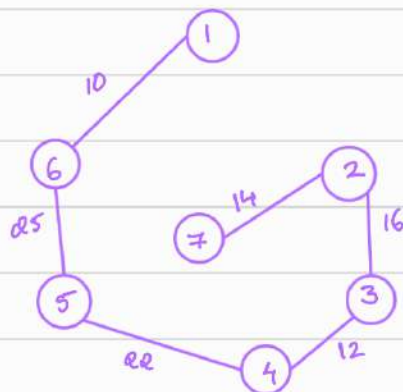


① No. of nodes should be connected by $(n-1)$ edges

② Acyclic graph

* NOTE:- Prim's can't be applied in a disconnected graph

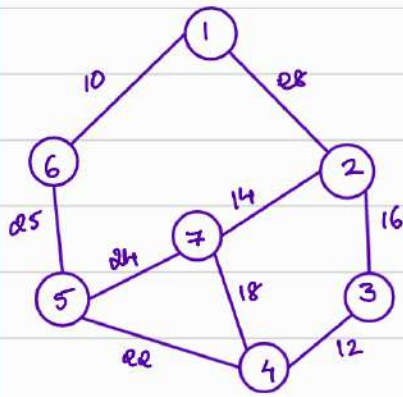
A:



⇒ Total (min.) cost of the Spanning Tree = 99

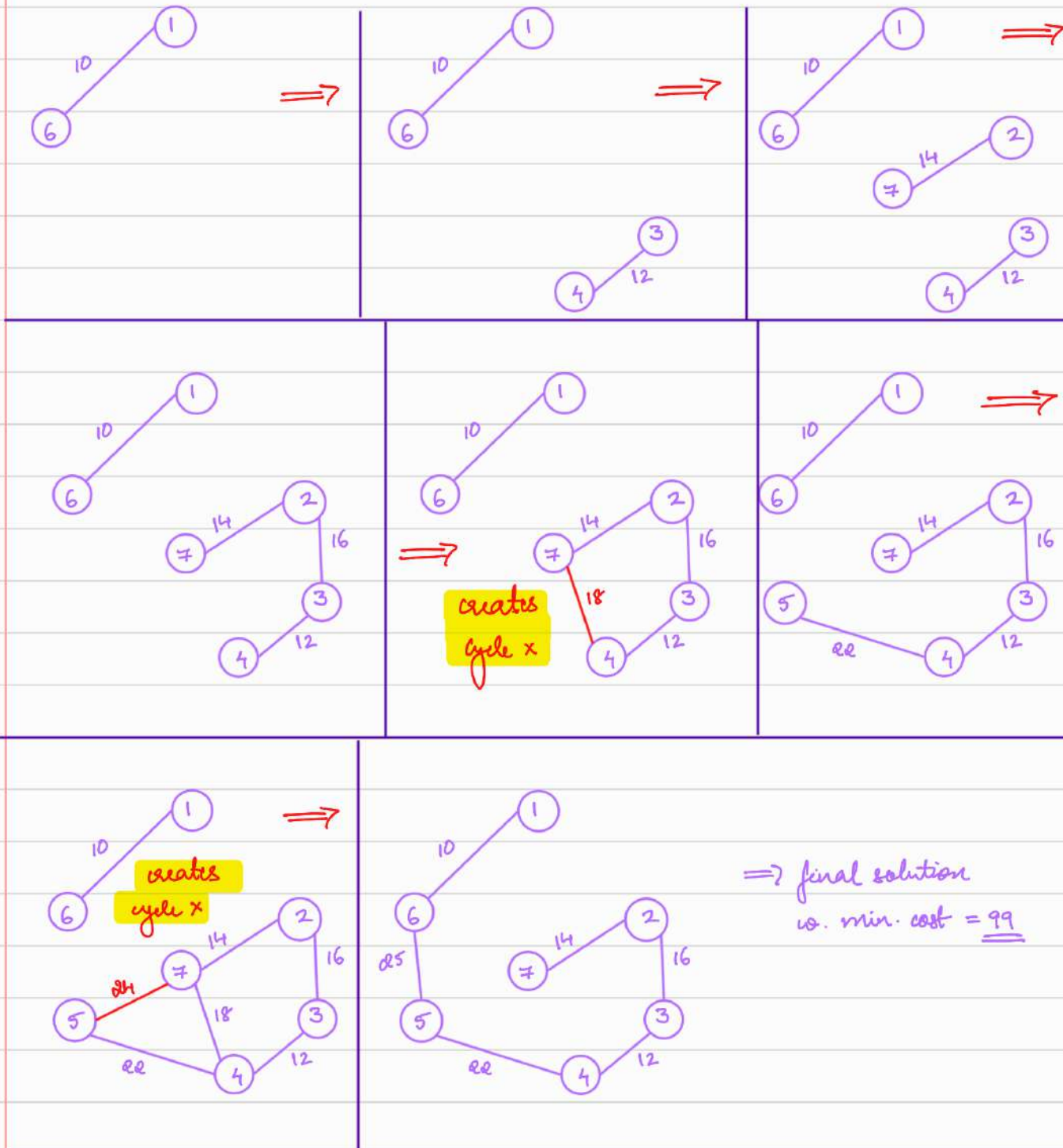
→ Kruskal's Algo.

Ex:

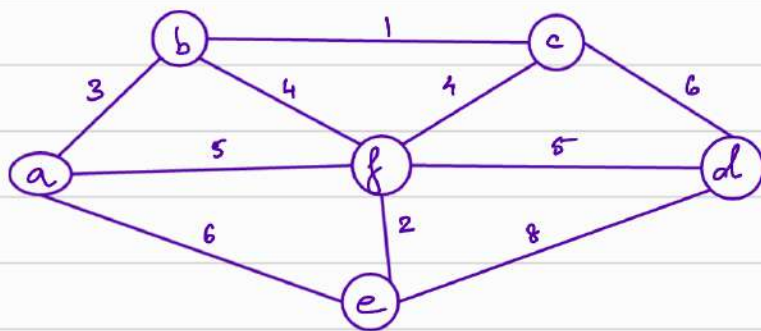


!! ${}^9C_6 - 3 = 81$ diff. possibilities to find MST

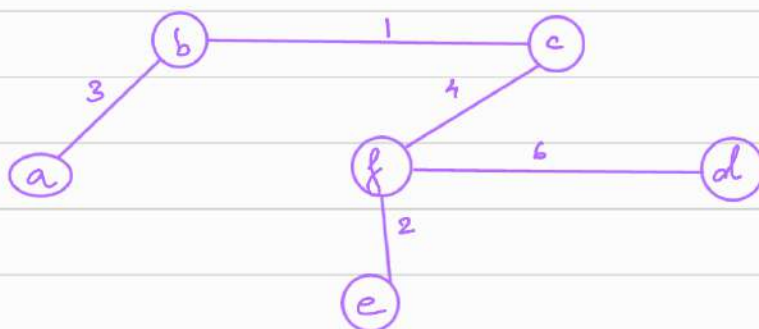
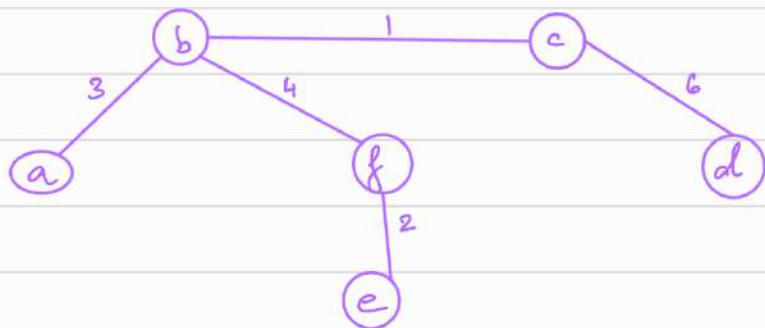
A:



Ex: Apply Prim's & Kruskal's to the graph below,



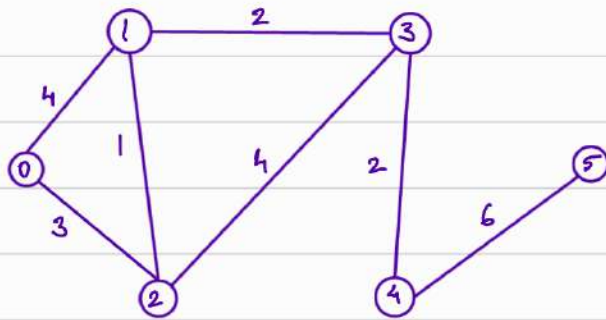
A:



2 Possibilities
using Prim's

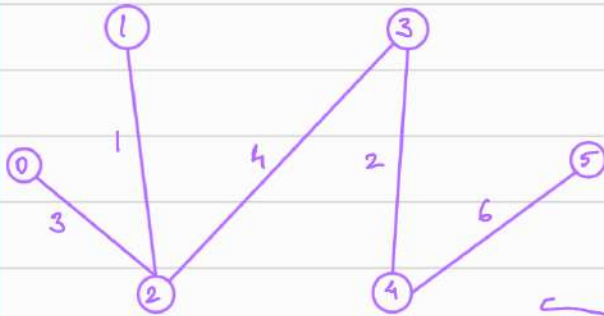
Portions
IA3 -
Hashing
IA4 -
Greedy Tech.

Ex:

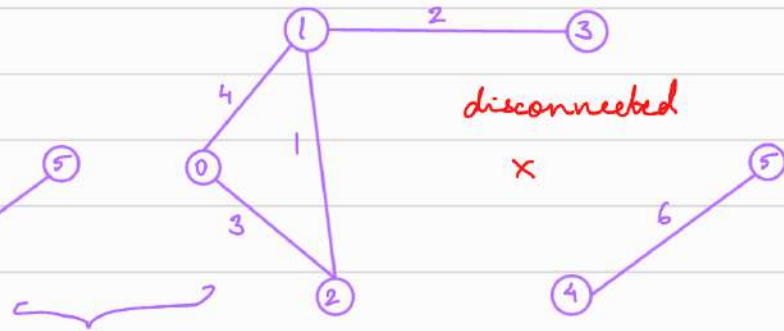


⇒ Apply Prim's Algo.

A:



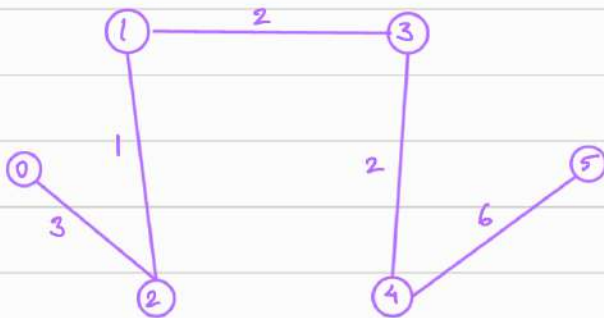
stays connected, Acyclic



disconnected

x

Applying Prim's Algo.

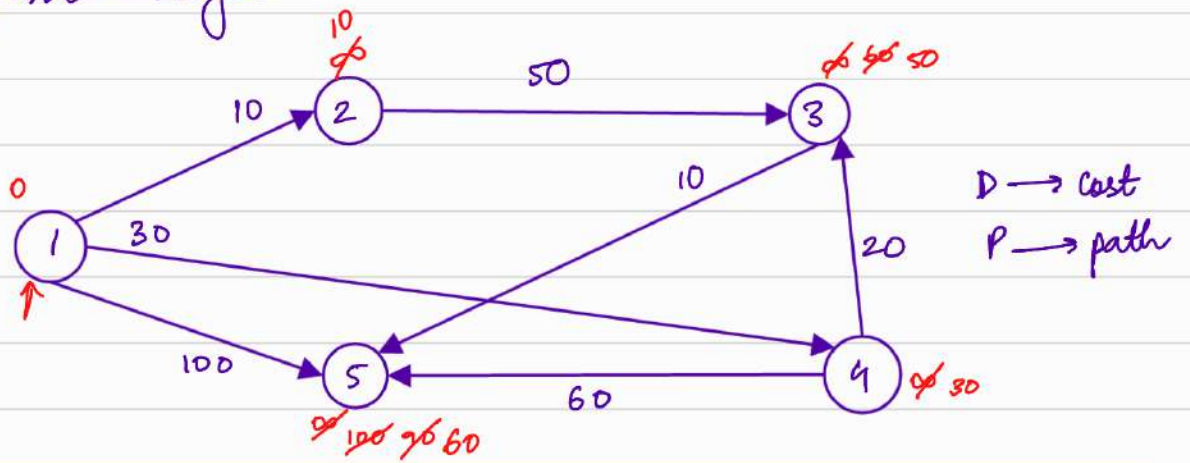


stays connected, Acyclic

⇒ Minimum Spanning Tree (MST)

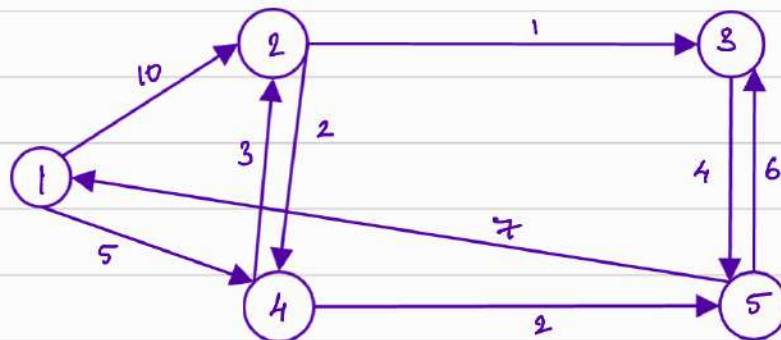
Dijkstra's Algo.

Eg:



Iteration	i	$D_2 P_2$	$D_3 P_3$	$D_4 P_4$	$D_5 P_5$	L
Initial	{1}	10, 1	$\infty, 0$	30, 1	100, 1	{2, 4, 5}
1	{2}	10, 1	<u>60, 2</u>	30, 1	100, 1	{4, 5, 3}
2	{3}	10, 1	<u>50, 4</u>	30, 1	<u>90, 4</u>	{3, 5}
3	{4}	10, 1	50, 4	<u>30, 3</u>	<u>60, 3</u>	{5}
4	{5}	10, 1	50, 4	30, 1	60, 3	{}

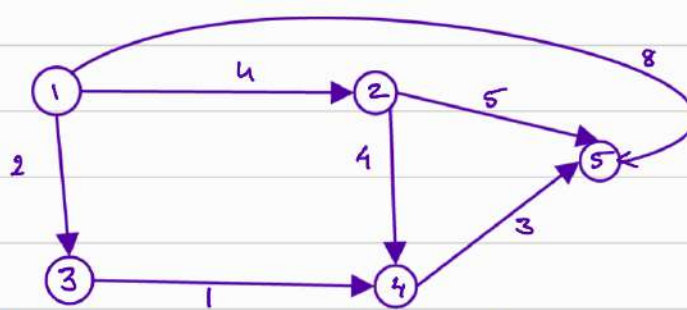
Q



A:

Iteration	i	$D_2 P_2$	$D_3 P_3$	$D_4 P_4$	$D_5 P_5$	L
Initial	{5}	10, 1	$\infty, 0$	5, 1	$\infty, 0$	{2, 4}
1	{A}	10, 1	11, 2	5, 1	14, 1	{3, 5}
2	{B}	10, 1	10, 2			
3	{C}					
4	{D}					

Q)



Apply Dijkstra's, source starting point is 1.

A:

Iteration	i	$D_2 P_2$	$D_3 P_3$	$D_4 P_4$	$D_5 P_5$	L
Initial	{1}	4, 1	∞, 0	∞, 0	8, 1	{2, 3, 5}
1	{2}	4, 1	∞, 1	4, 2	5, 2	{4, 5}
2	{3}	4, 1	2, 1	3, 1	4, 3	{4, 5}
3	{4}	4, 1	2, 1	3, 1	3, 4	{5}
4	{5}	4, 1	2, 1	3, 1	3, 4	{ }

ac/3/26



Huffman Coding

Ex:

B C C A B B D D A E C C B B A E D D C C

A:

no. of char(s) = 20

1 Byte = 8 bits; each char takes = 8 bits

total no. of bits = $20 \times 8 = 160$ bits

Unique char	no. of times repeated frequency	
A	3	since 5 vals (---) we take 3 bits for % combinations
B	5	
C	6	
D	4	
E	2	

Unique char frequency

A	3
B	5
C	6
D	4
E	2

Suppose assigning

000
001
010
011
100

$$3 \times 3 = 9$$

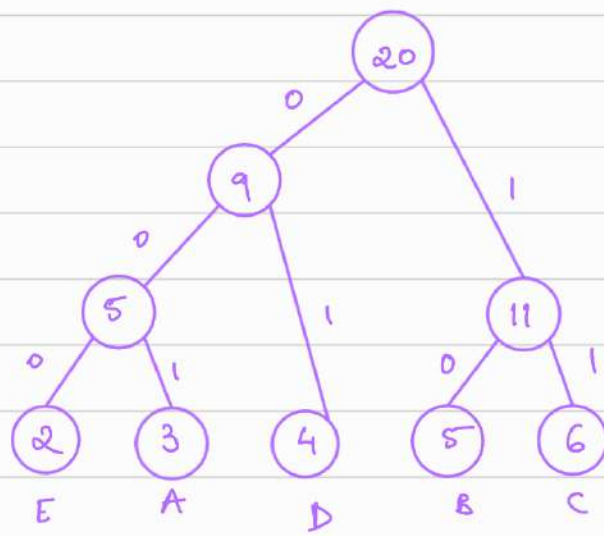
$$3 \times 5 = 15$$

$$6 \times 3 = 18$$

$$4 \times 3 = 12$$

$$2 \times 3 = 6 / 60 \text{ bits}$$

$$\begin{aligned}
 &= 60 + (8 \times 5) \\
 &+ (3 \times 5) \rightarrow \text{sa} \\
 &= 115 \text{ bits} \\
 &\text{fixed coding}
 \end{aligned}$$



instead of tabular representation, leaf node method of tree.
 * All RHS of the leaf node is 0 & LHS is 1

Now, from the tree we get,

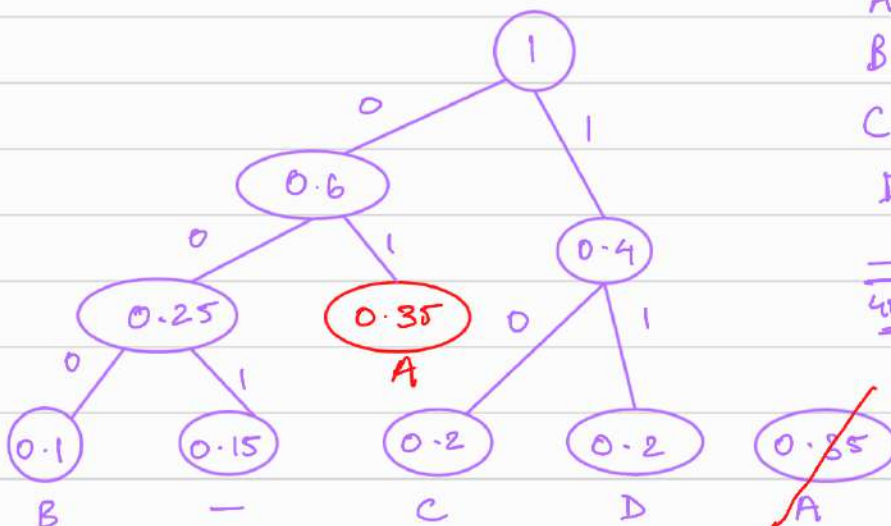
	no. of bits	freq. $\times$ no. of bits
A	001	$(3 \times 3 = 9)$
B	10	$(5 \times 2 = 10)$
C	11	$(6 \times 2 = 12)$
D	01	$(4 \times 2 = 8)$
E	000	$(2 \times 3 = 6)$
	<u>40 bits</u>	<u>45 bits</u>
	<u>97 bits</u>	

Q)

char	A	B	C	D	—
freq	0.35	0.1	0.2	0.2	0.15

A:

char	A	B	C	D	—
freq	0.35	0.1	0.2	0.2	0.15
	<u>= 1</u>				



A	→ 01	(2 × 0.35)
B	→ 000	(3 × 0.1)
C	→ 10	(2 × 0.2)
D	→ 11	(2 × 0.2)
—	→ 001	(3 × 0.15)
<u>40 bits</u>	<u>12 bits</u>	<u>02.25 bits</u>
<u>54.25 bits</u>		

∴ char	A	B	C	D	—
freq	0.35	0.1	0.2	0.2	0.15
codeword	11	100	00	01	101

$$\text{Compression Ratio} = \frac{(3 - 2.25)}{(3 \times 100)} = \underline{\underline{25\%}}$$

→ Optimal tree problem

→ fixed leng. coding (bit string len = m, Eg:- ASCII Code)

→ Variable leng. coding (LHS = 0, RHS = 1 | David Huffman)

Q) Apply Huffman Coding for,

char: ABCDEBCDECEDEEE

A:

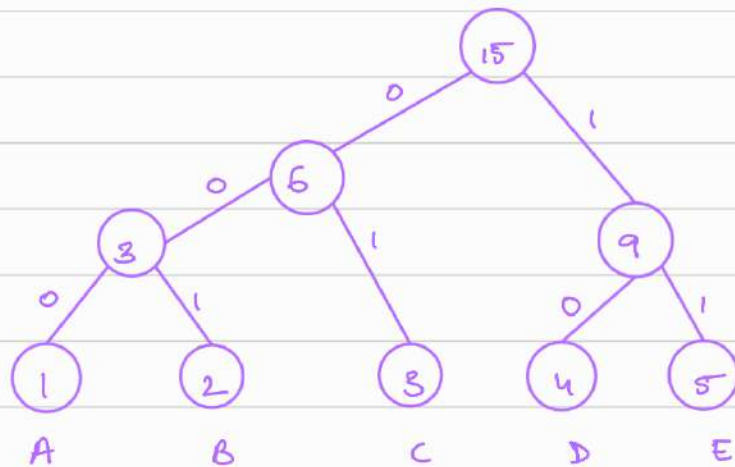
A — 1

B — 2

C — 3

D — 4

E — 5



A — 000 (3x1) 3

B — 001 (3x2) 6

C — 01 (2x3) 6

D — 10 (2x4) 8

E — 11 (2x5) 10

40 bits 12 bits 33 bits

85 bits

∴ char	A	B	C	D	E
freq	1	2	3	4	5
codeword					

28/5/27

$$\text{Compression ratio} = \left(\frac{\quad}{\times 100} \right) =$$

N, P, and NP Problems

- There are many problems for which the answer is a Yes or No. These types of problems are known as decision problems. (Decision Problem)
- A decision problem X is said to be in class P, if it can be solved using deterministic Turing machine in polynomial time. (class P)
 - ↳ Eg:- Sorting, Searching, finding shortest path etc.
- Non Deterministically Polynomial (NP)
 - A decision problem X if can be solved in polynomial time using non-deterministic Turing machine

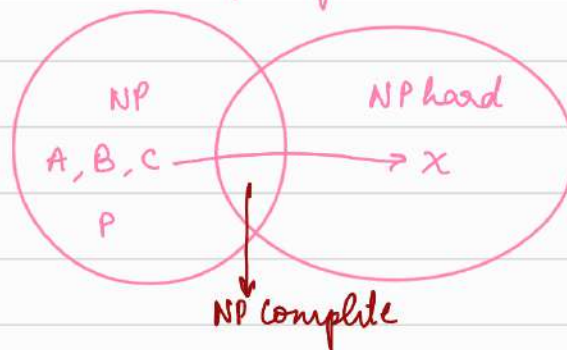
Eg:- Decision version of 0/1 Knapsack problem, TSP etc.

→ P vs. NP

- Clearly class P is a subset of NP
- But whether NP is a subset of P is not a big as. problem.

→ NP Complete

- A problem is NP-complete if it is both NP-hard in NP.



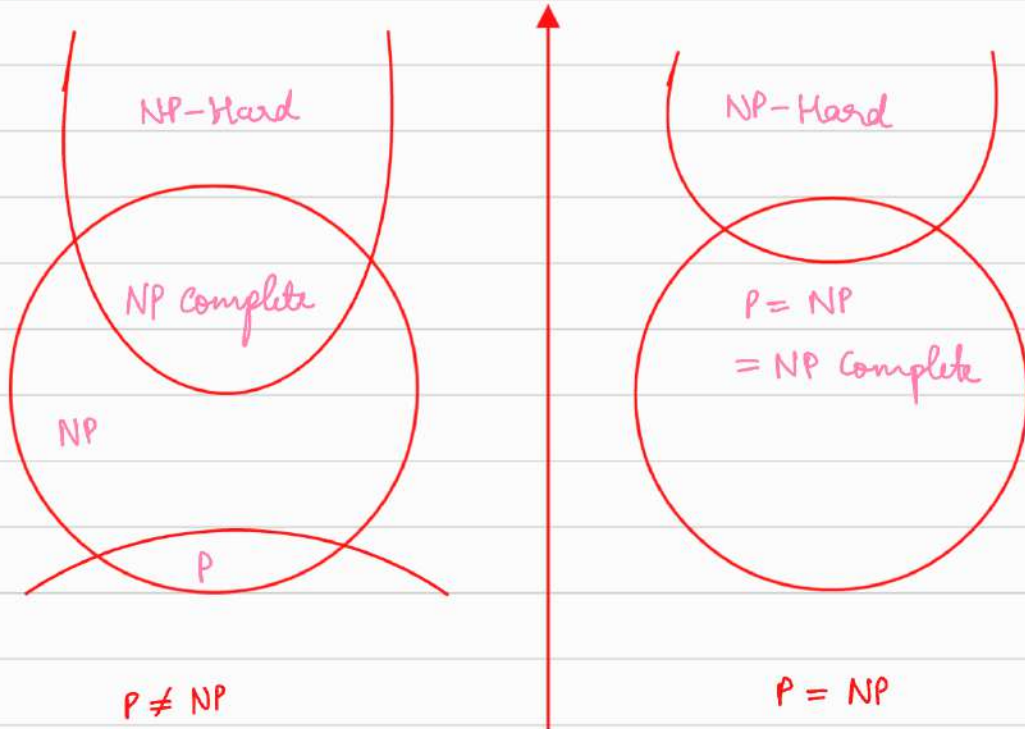
→ NP-Hard

Polynomial time reductions:-

→ If problem A can be poly-time reduced to problem B, then it stands to reason B is at least as hard as A.

→ A problem is NP-hard if every problem in NP can be poly-time reduced to it

→ $P = NP$ and $P \neq NP$ Scenario



8/13/26

→ Backtracking

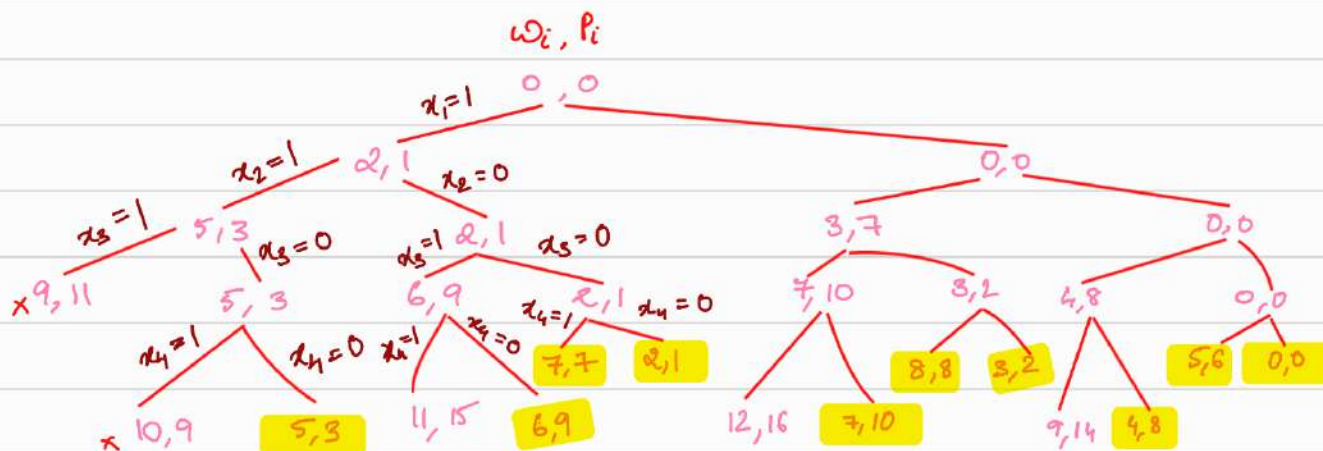
→ state space tree (0/1 Knapsack Problem) [Maximization Problem]

Q)

	x_1	x_2	x_3	x_4
item	1	2	3	4
w_i	2	3	4	5
p_i	1	2	8	6

Capacity, $C = 8$

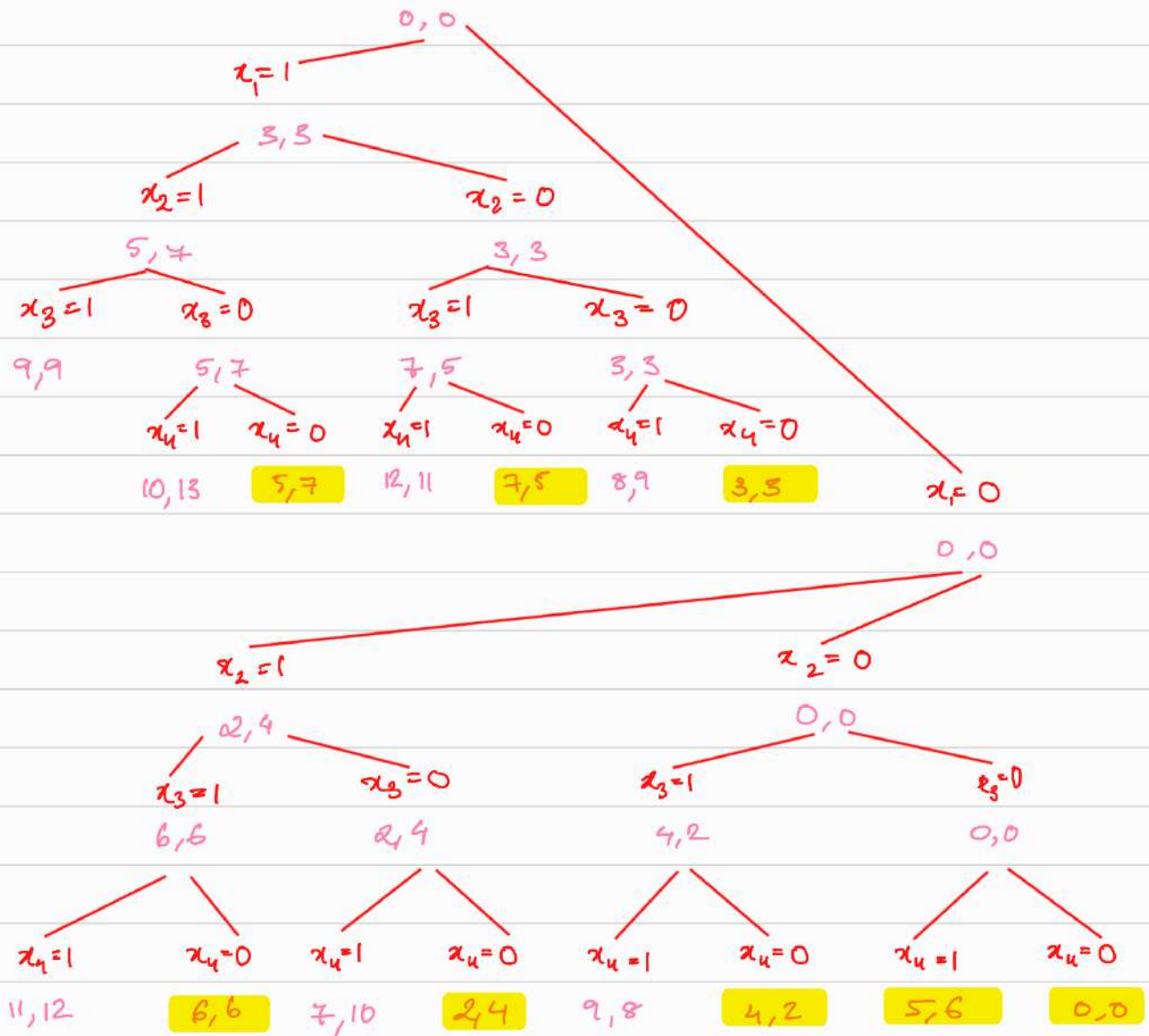
A:



Ex:

	x_1	x_2	x_3	x_4	
item	1	2	3	4	
w_i	3	2	4	5	$C = 8$
p_i	3	4	2	6	

5:



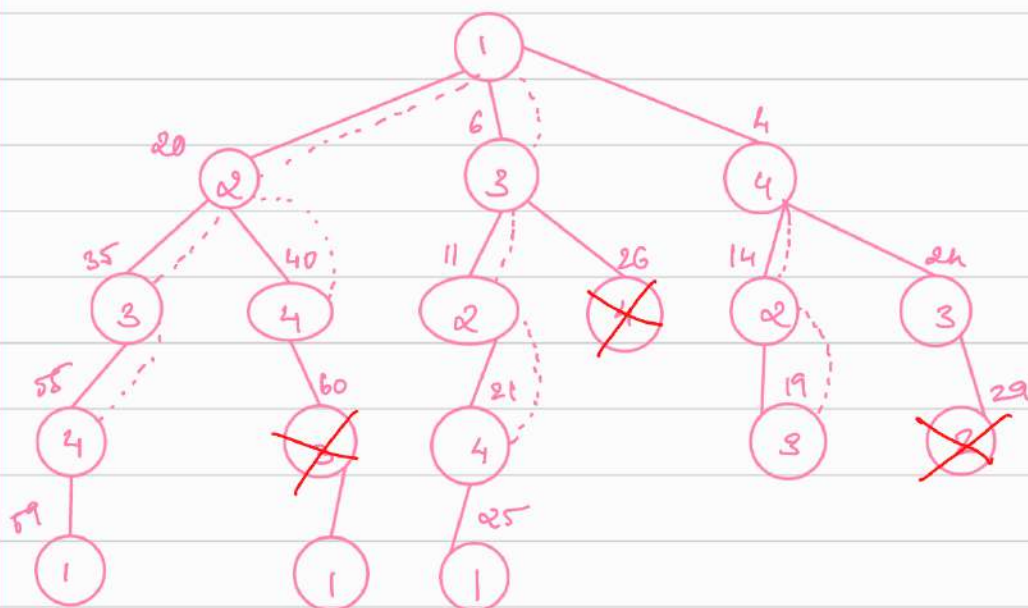
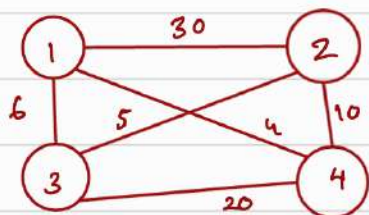
H.W
Ex:

	x_1	x_2	x_3	x_4	x_5
item	1	2	3	4	5
w_i	4	3	5	6	2
P_i	$9/4$	$7/3$	$10/5$	$9/6$	$2/2$
	1.3	2.3	2	1.5	1

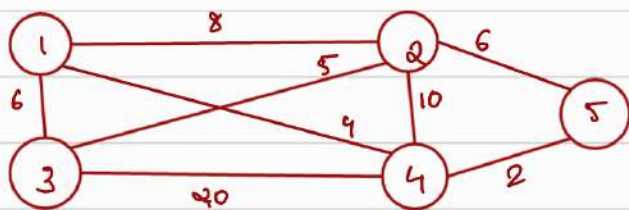
; $C=18$

→ Travelling Salesman (State Space Tree: Minimization Problem)

Q)



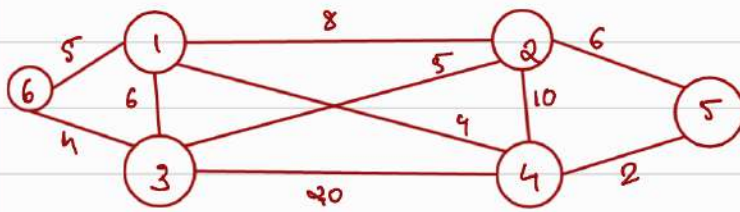
Q)



A:

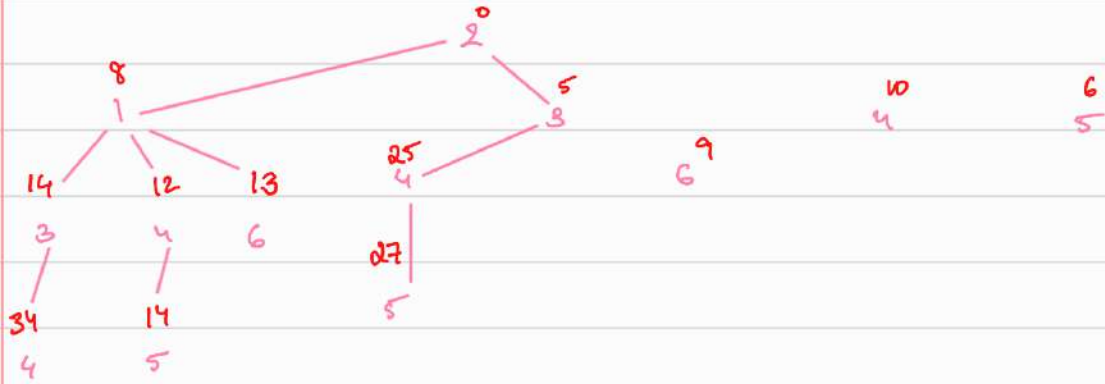


Q2

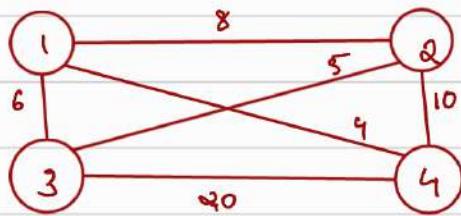


; start from node 2.

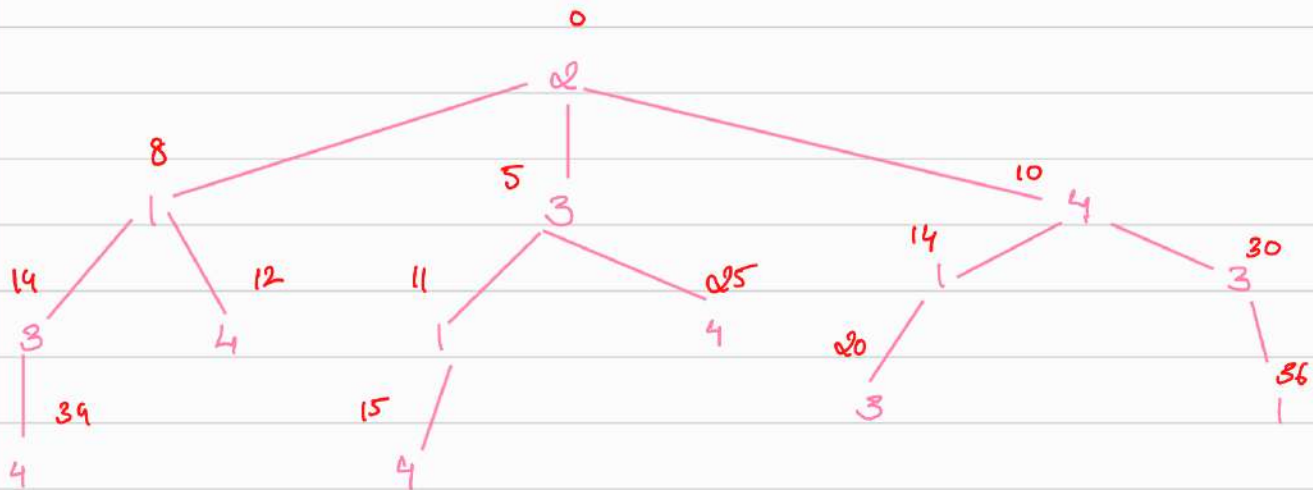
A:



Q2



; start from node 2.



4/4/26

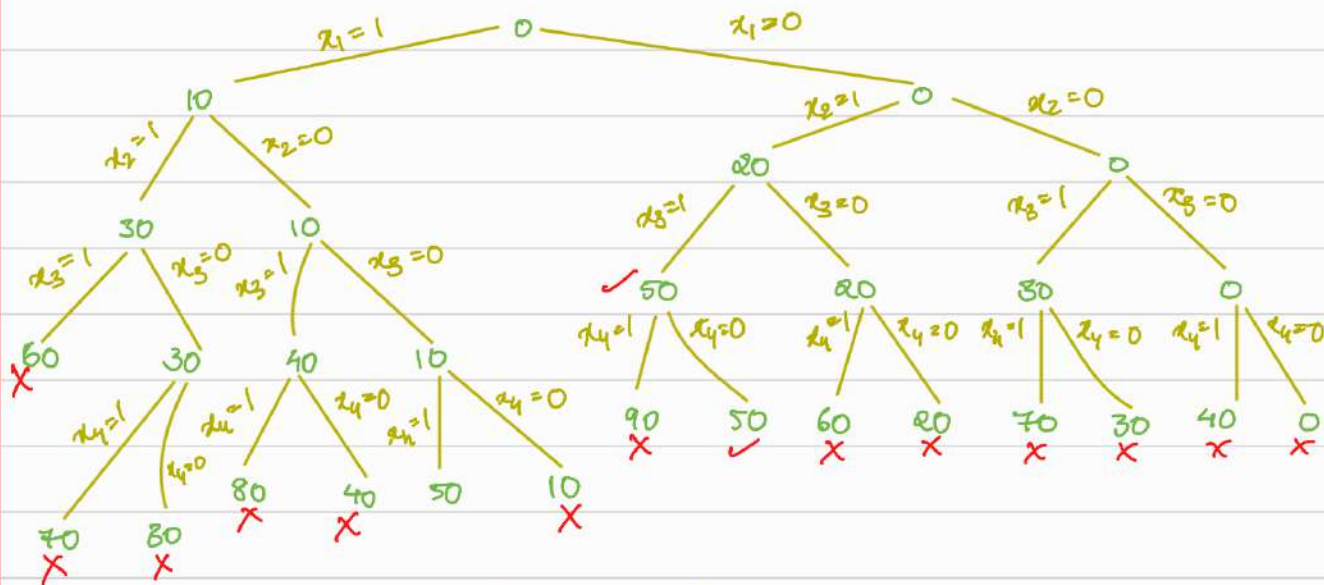
12.40 - 1pm : Data IA4 (Quiz)

Sum of Subsets

Q) $W = \{10, 20, 30, 40\}$; $sum = 50$; $n = 4$
 $x_1 \ x_2 \ x_3 \ x_4$

NOTE: - If node exceeds sum or is more, kill the node.

A:

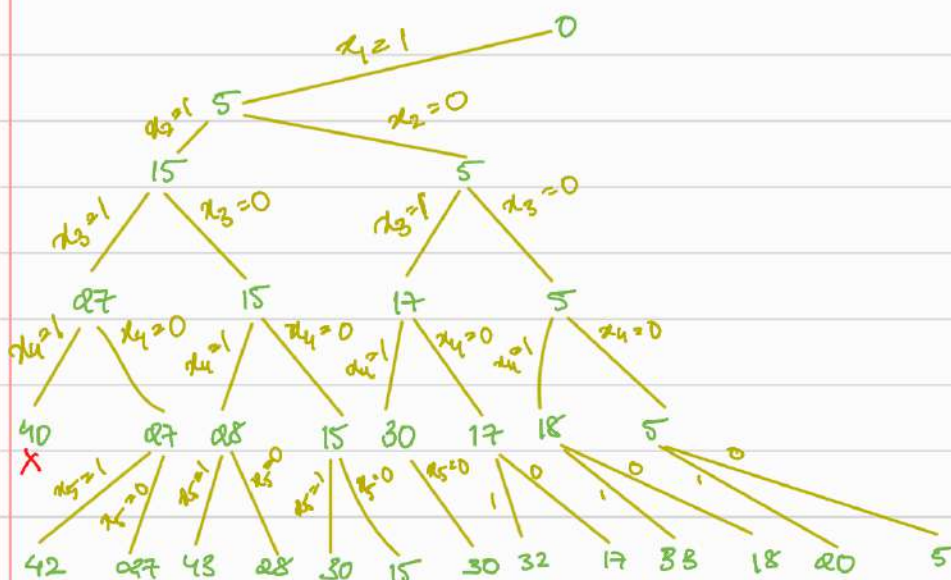


paths :- 0110

1001

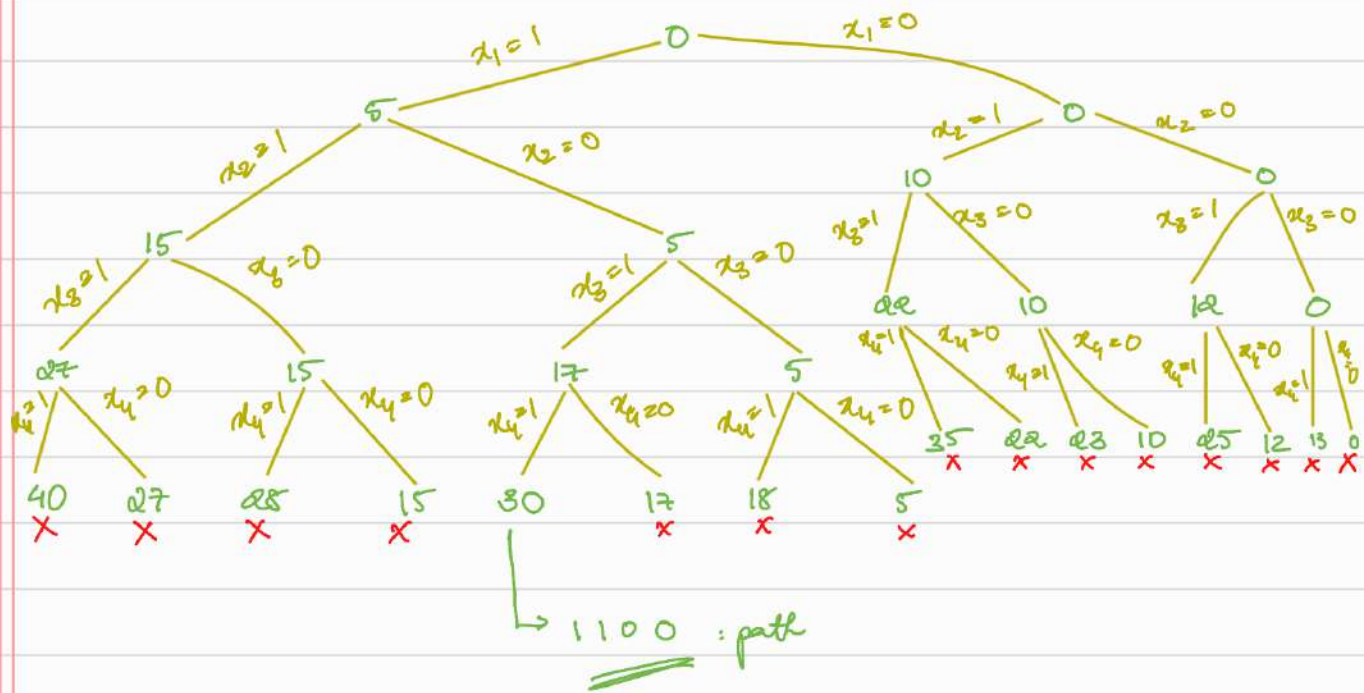
Q) $W = \{5, 10, 12, 13, 15, 18\}$; $n = 6$; $sum = 80$
 $x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6$

A:



Q)

$$\omega = \{5, 10, 12, 13\}; n = 4; \text{sum} = 30$$



8/4/25

Branch & Bound

→ o/ Knapsack Problem (Maximization; UB)

Q	item	1	2	3	4	
	w	4	7	5	3	
	v	40	42	25	12	
	v/w					

; C = 10

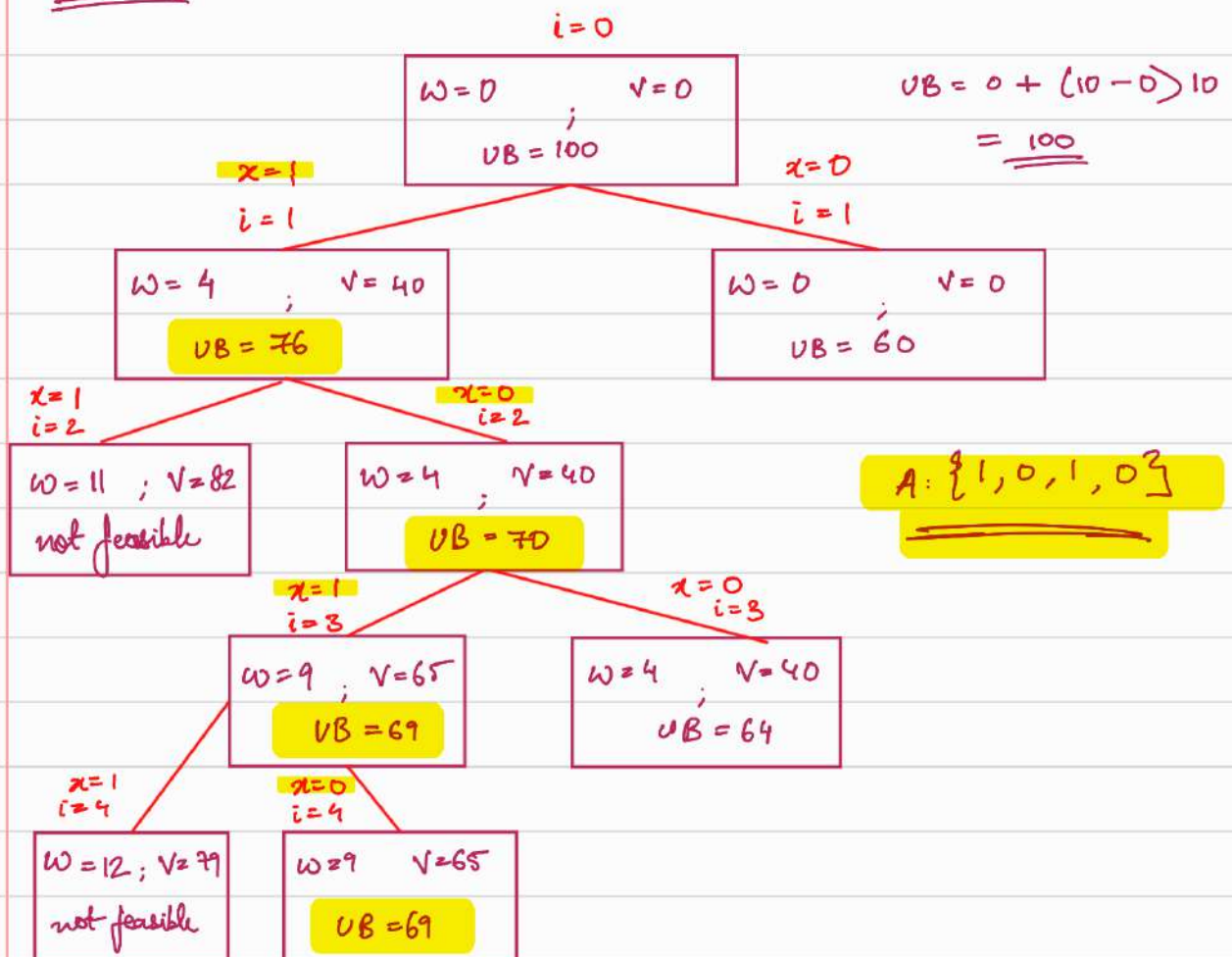
Formula:-

$$\text{Upper Bound} = V + (C - w) \left[\frac{v_{i+1}}{w_{i+1}} \right]$$

A:

item	1	2	3	4
w	4	7	5	3
v	40	42	25	12
v/w	10	6	5	4

State space tree



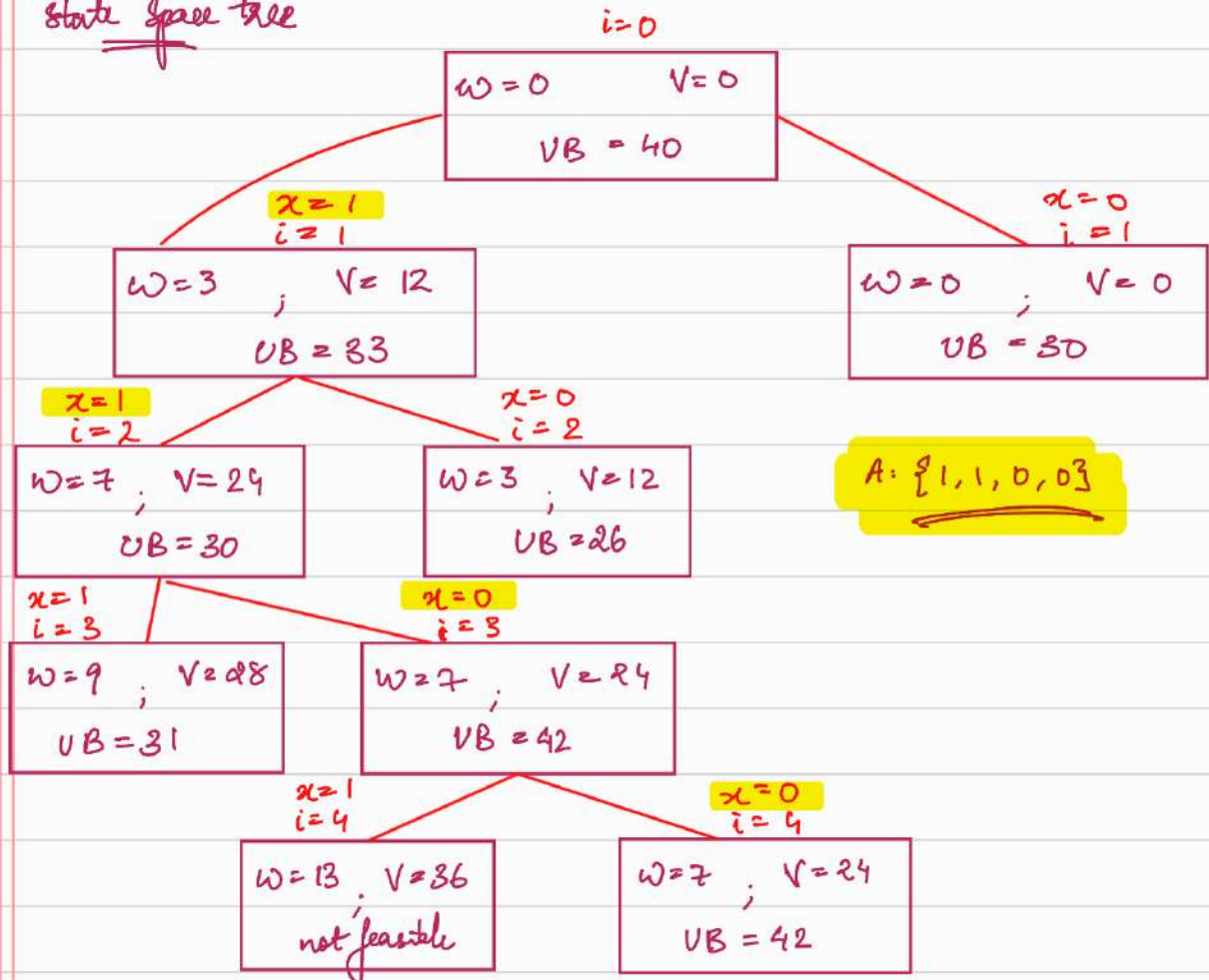
Q)

item	1	2	3	4
w	3	4	2	6
v	12	12	4	12
v/w	4	3	2	6

$$C=10$$

A:

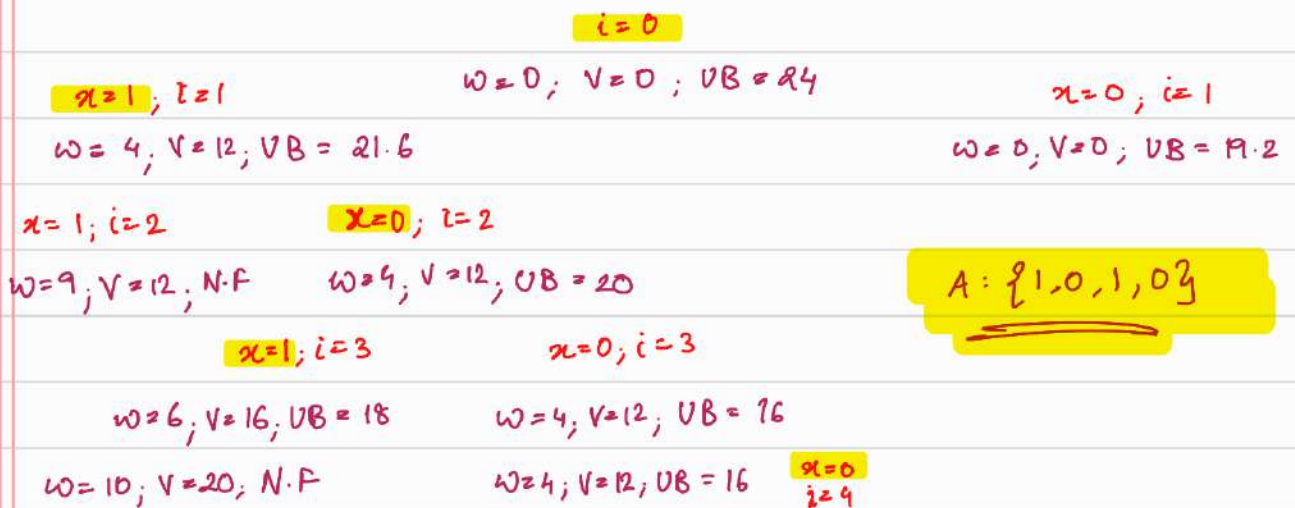
state space tree



Q)

item	1	2	3	4
w	4	5	2	4
v	12	12	4	4
v/w	3	2.4	2	1

$$C=8$$



a)

item	1	2	3	4
w	2	4	6	9
v	10	10	12	9
v/w	5	2.5	2	1

$$C = 15$$

A:

$$i = 0$$

$$w = 0; v = 0; UB = 75$$

$$x = 1; i = 1$$

$$x = 0; i = 1$$

$$w = 2; v = 10; UB = 42.5$$

$$w = 0; v = 0; UB = 87.5$$

$$x = 1; i = 2$$

$$x = 0; i = 2$$

$$w = 6; v = 20; UB = 88$$

$$w = 2; v = 10; UB = 36$$

$$x = 1; i = 3$$

$$x = 0; i = 3$$

$$w = 8; v = 22; UB = 29$$

$$w = 2; v = 10; UB = 23$$

$$x = 1; i = 4$$

$$x = 0; i = 4$$

$$w = 17; v = 31; N.F$$

$$w = 8; v = 22; UB = 29$$

$$\Rightarrow A: \{1, 0, 1, 0\}$$

9/4/26 Q)

item	1	2	3	4
w	6	3	4	5
V	12	12	12	12
v/w	2	3	4	2.4

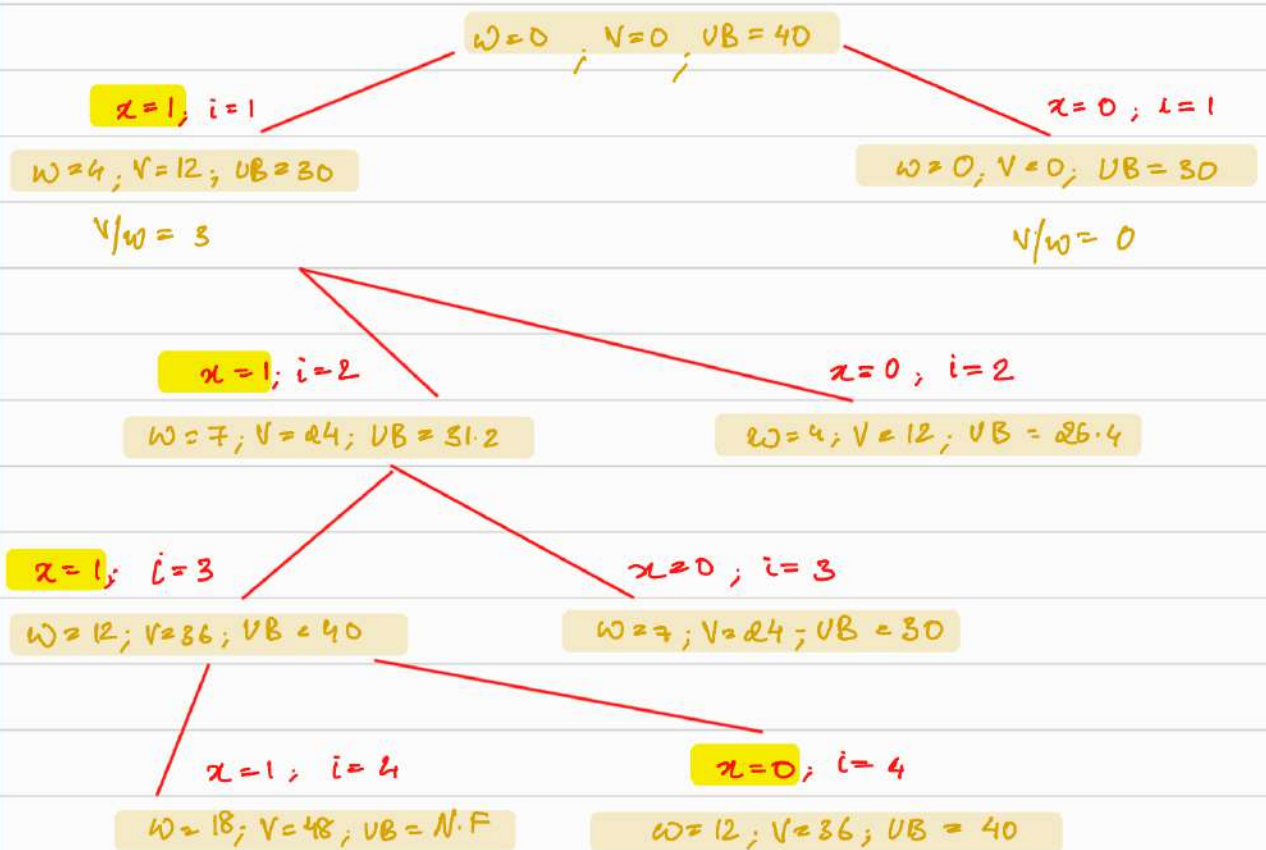
won't give profitable value $\Rightarrow$ sort it in desc. order

A:

item	3	2	4	1
w	4	3	5	6
V	12	12	12	12
v/w	4	3	2.4	2

→ descending order (sorted order)

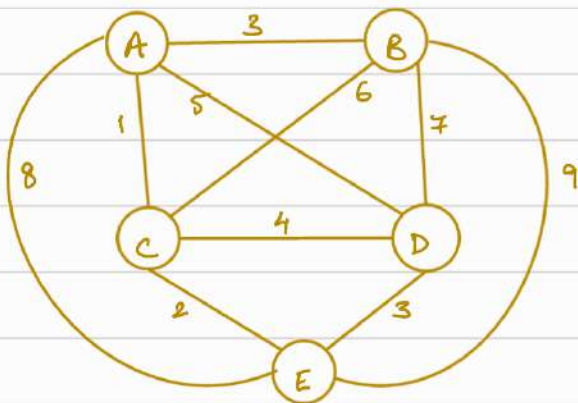
$$UB = 0 + (10 - 0)(4) = \underline{40}$$



$$A = \{0, 1, 1, 1\} \times \rightarrow A = \underline{\underline{\{0, 1, 1, 0\}}}$$

→ Travelling Salesman Problem (Minimization; LB)

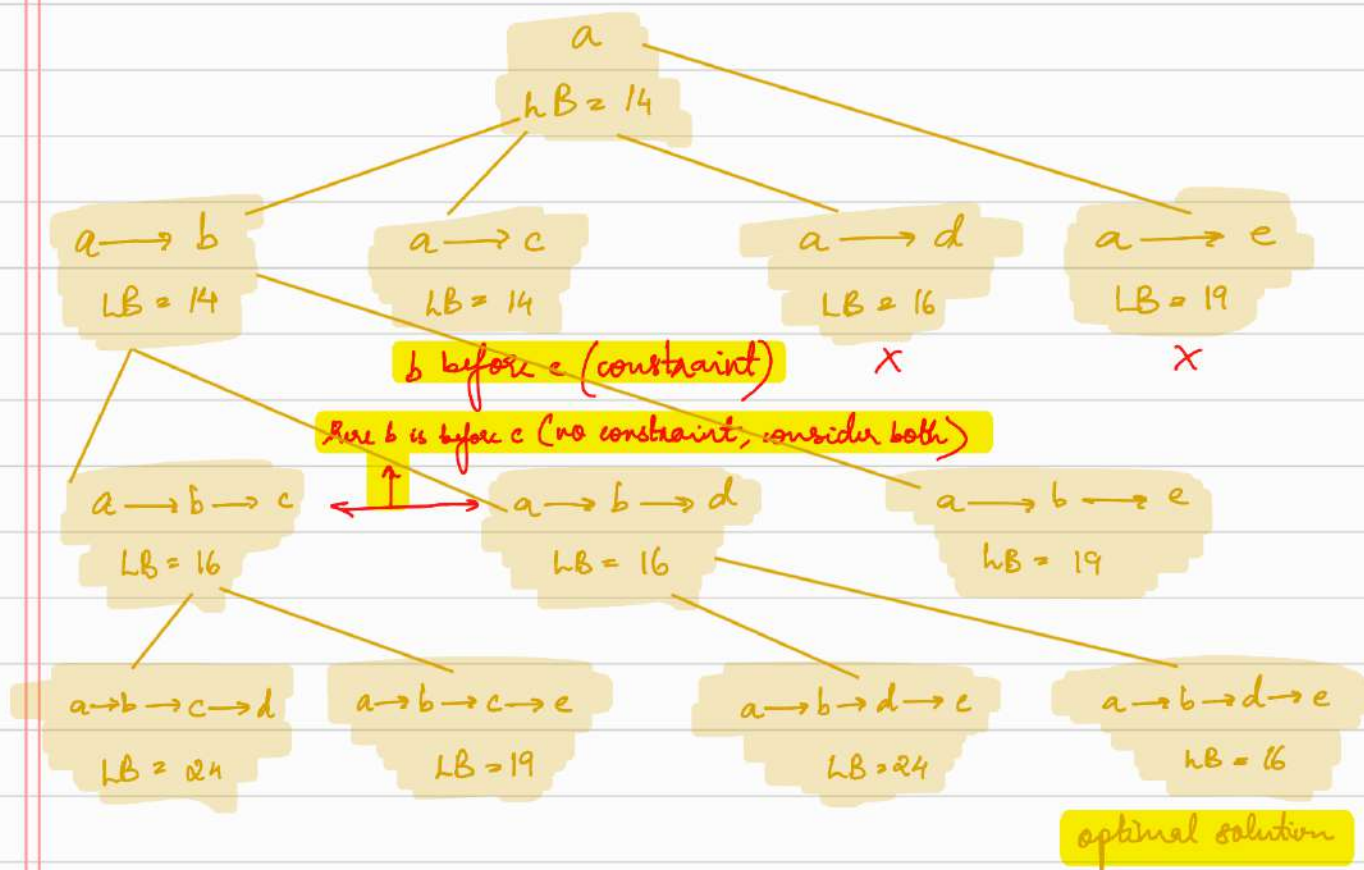
Q)



$$LB = \frac{3}{2}$$

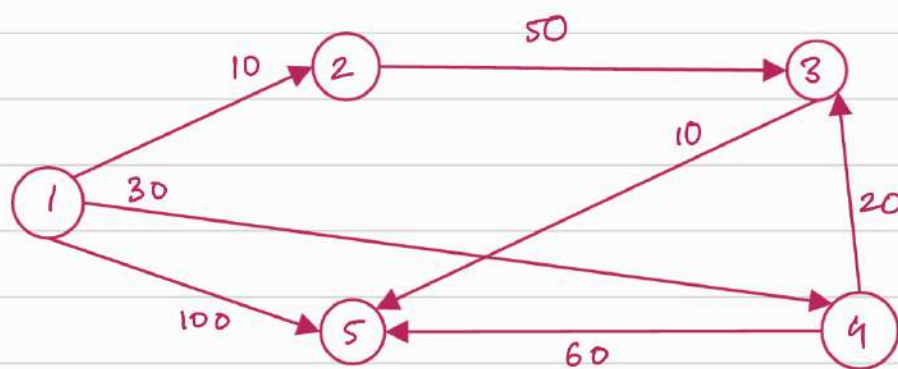
Ans: $LB = \frac{8}{2} = \frac{(AB+AC) + (AB+BC) + (AC+CE) + (DE+CD) + (CE+DE)}{2} = \frac{28}{2} = 14$

State Space tree



Single Source shortest Path Problem: Dijkstra's Algorithm

Q)



D → Cost
P → path

A:

Iteration	i	D ₂ P ₂	D ₃ P ₃	D ₄ P ₄	D ₅ P ₅	h
Initial	{1}	10, 1	∞, 0	30, 1	100, 1	{2, 4, 5}
1	{2}	10, 1	60, 2	30, 1	100, 1	{4, 5, 3}
2	{3}	10, 1	50, 4 1-4-3	30, 1	90, 4 1-4-5	{5}
3	{4}	10, 1	50, 4	30, 1	60, 3 1-4-3-5	{5}
4	{5}	10, 1	50, 4	30, 1	60, 3	{1}